Vietnam National University

Ho Chi Minh city University of Technology

Faculty of Computer Science and Engineering

ELECTRONIC COMMERCE
Database Systems(CO2014) - CC09

Report Date: 07/10/2025 - Semester 251

Lecturer: NGUYEN MINH TAM

Student name	Student ID	Email
Đỗ Đăng Khoa	2352554	khoa.do250805@hcmut.edu.vn
Trần Liên Khang	2352504	khang.tran0209@hcmut.edu.vn
Tào Nguyễn Quang Khang	2352499	khang.tao206bkuhcm@hcmut.edu.vn
Nguyễn Phước Quý Bảo	2352100	bao.nguyen090105@hcmut.edu.vn

Table of Contents

List of Figures

List of Tables

1 Introduction

In the era of digital commerce, online marketplaces have become an essential medium for modern retail activities. Platforms such as Shopee demonstrate the effectiveness of connecting millions of buyers and sellers through a unified digital infrastructure, providing convenience, scalability, and automation in commercial transactions. Inspired by this model, the proposed system aims to design a database to support a multi-role e-commerce platform in which individual customers can browse and purchase products, merchants can manage storefronts and process orders, and system administrators can supervise platform operations.

This report focuses on analyzing the functional requirements of such a platform and designing a suitable database structure to support core activities, including user account management, product categorization, shopping cart handling, order processing, voucher application, shipping coordination, and post-purchase feedback. The objective is to construct a logical data model that ensures consistency, scalability, and extensibility, laying the foundation for future development of a complete e-commerce application.

2 Application Description

2.1 Research related applications/systems

The selected reference application is **Shopee**, one of the leading E-Commerce platforms in Vietnam. The website can be accessed at https://shopee.vn. **Shopee** connects customers and sellers in an online marketplace, providing a comprehensive system to browse, purchase, and manage products. Its business process begins with account registration, where users can sign up as customers or sellers. Customers browse products organized in hierarchical categories, select variants of products such as size or color, add items to a shopping cart, and proceed to checkout with payment and shipping options. Sellers manage shop profiles, upload and update product listings, track inventory, and monitor customer reviews to improve service quality.

A key feature of **Shopee** is the multi-shop order system, where a single order can include products from multiple sellers, each tracked individually in the backend. Another important element is the voucher system, which applies discounts based on conditions such as minimum order value or expiration dates. **Shopee** also integrates a review and rating system, ensuring that verified purchases can leave feedback on the product and the store, which helps improve trust and transparency.

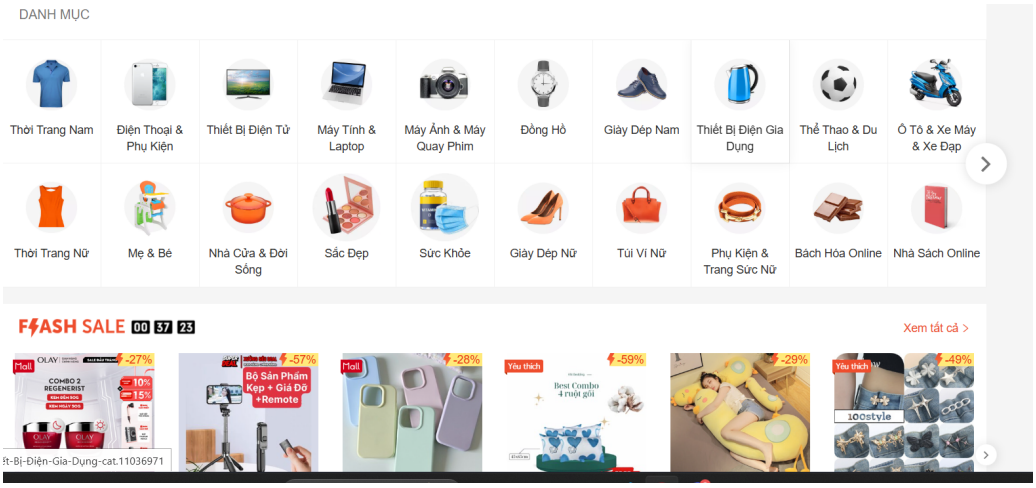

Figure 1: Homepage interface of **Shopee** when accessing the website for the first time

The homepage highlights featured campaigns, recommended products, and navigation menus, such as categories, products and flash sales. This layout indicates that **Shopee** focuses heavily on visual-driven marketing and personalized product discovery.

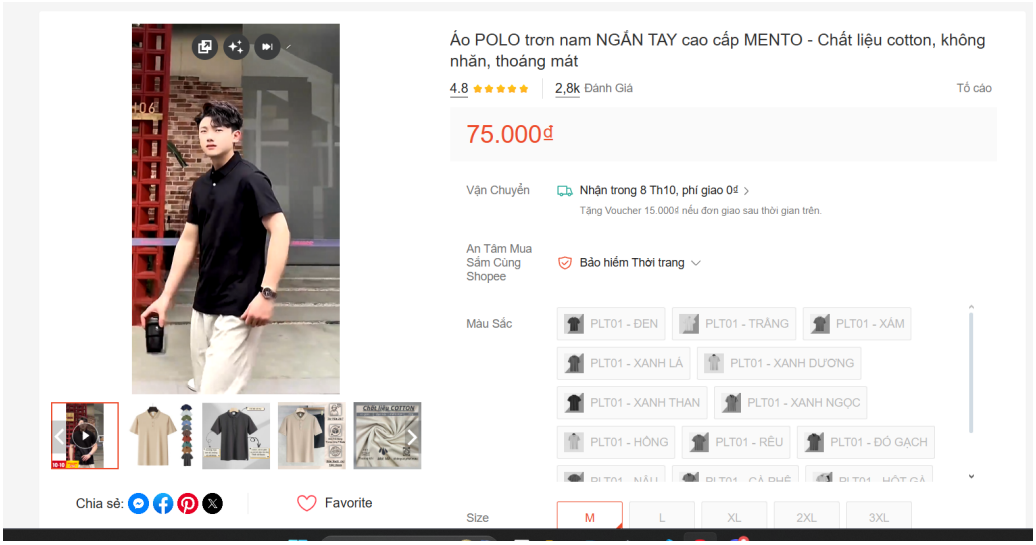

Figure 2: Product detail page with variant selection (size, color), price display, and image gallery

This screen demonstrates the purchasing flow, where users can select a specific **product variant**, adjust quantity, add to cart or directly place an order. Essential information such as ratings, shipping options, and shop reputation is also displayed to support decision-making.

2.2 Description of the Proposed System

2.2.1 General Description

The proposed system is an e-commerce platform inspired by Shopee, designed to support online retail activities between buyers and sellers. It enables individuals and businesses to create selling

accounts, publish products, and manage orders, while customers can browse, purchase, and evaluate products in a seamless digital marketplace.

Types of Users: **Customer**: A regular user who browses products, adds items to their shopping cart, places orders, applies vouchers, selects shipping methods, and provides product or shop reviews after successful purchases.

Shop: A seller entity responsible for managing product listings, defining product variants, monitoring stock levels, handling orders, and receiving feedback from customers.

Admin: A privileged system user who supervises overall platform operations, manages user accounts, resolves disputes, and moderates inappropriate content when necessary.

Main System Functions: For **Customers**: Browse products by category, search and filter based on attributes, add items to cart, update quantities, place orders with preferred shipping and payment methods, apply vouchers, and submit verified reviews for purchased products.

For **Shops**: Register as sellers, manage store profile, create product listings with multiple variants, track inventory, process incoming orders, and analyze ratings and feedback to improve service quality.

For **Admins**: Oversee account registration activities, suspend violating accounts, manage system-wide configurations such as shipping options and voucher policies, and intervene in reported cases of fraud or misconduct.

2.2.2 Detailed Description of Entity Types, Attributes and Relationships

ACCOUNT Entity The system maintains a centralized **ACCOUNT** entity to identify every registered user. Each account is uniquely defined by an **AccountID** (auto-incremented primary key) and stores essential authentication details such as **Email** and **Password**. Personal identification is represented through the attribute **FullName**, while **Phone** stores the primary contact information and must follow Vietnamese phone format (10 digits). The **Role** field classifies each account as Customer, Shop, or Admin, ensuring proper access control. The attribute **Status** maintains activation state with values 'Active' or 'Ban' for moderation purposes. **CreatedAt** records the registration timestamp automatically.

CUSTOMER Entity The **CUSTOMER** entity is a specialization of ACCOUNT and represents end-users participating in purchasing activities. Each customer is referenced by a distinct **CustomerID** (same as AccountID) and an auto-generated **CustomerCode** with 'CUST' prefix (CUST00001, CUST00002, ...). Customers have an **Address** for default shipping destination and **AddPhone** for additional contact. The system maintains **TotalSpent** and **TotalOrder** as derived attributes to support customer ranking and loyalty programs. Each CUSTOMER owns exactly one **CART**.

SHOP Entity The **SHOP** entity models registered merchants. Each shop is assigned a unique **ShopID** (same as AccountID) and an auto-generated **ShopCode** with 'SHOP' prefix (SHOP00001, SHOP00002, ...). A shop operates under a **ShopName** and has a **ShopPhone** for business inquiries. The **AddressShop** identifies its operational base, while **Rating** reflects aggregated customer

feedback (0-5 scale). The **ShopStatus** attribute can be 'Open', 'Temporarily Close', or 'Closed'. A single SHOP may publish multiple **PRODUCTS**.

ADMIN Entity The **ADMIN** entity represents internal supervisory staff. Each admin is identified by an **AdminID** (same as AccountID) and is linked to an **AccountID**. The **Role** field specifies the admin's level of access (Core, Moderator, or Support), ensuring proper authorization. Unlike other roles, admins do not engage in transactions but retain full access rights for system governance. **Note** field allows additional admin information.

CATEGORY Entity The **CATEGORY** entity organizes products into a navigable structure. Each category has a unique **CategoryID** and a descriptive **CategoryName** (max 100 characters). Categories support hierarchical organization through **ParentCategoryID**, which can be null for root categories. This recursive 1:N relationship enables multi-level product browsing.

PRODUCT Entity The **PRODUCT** entity represents commercial goods listed by shops. Each product is uniquely defined by a **ProductID**, belongs to exactly one SHOP through **ShopID**, and must be classified under a CATEGORY via **CategoryID**. Descriptive attributes include **ProductName**, **Description**, **Image**, and **CreatedAt**. The attribute **Status** specifies whether the product is 'In stock' or 'Out of stock'. **Rating** is auto-calculated from customer reviews. Products serve as parent entities for **PRODUCTITEM** (variants).

PRODUCTITEM Entity **PRODUCTITEM** is a weak entity that models purchasable variations of a product. Each item is uniquely identified by **ItemID** and belongs to a specific **ProductID**. Each variant contains attributes such as **Color**, **Type** (size, configuration), **Price**, **Stock**, and optionally **ImageURL** for representation. Variants enable customers to differentiate between sizes, configurations, or styles within the same product family. **ShopID** is stored for quick reference to the selling shop.

CART Entity The **CART** entity holds temporary selections before checkout. Each cart is exclusively owned by one CUSTOMER and distinguished by **CartID**. The relationship between CART and PRODUCTITEM is modeled as a many-to-many association through **CARTITEM**, enriched with the attribute **Quantity**. **CreatedAt** and **UpdatedAt** track cart lifecycle.

CARTITEM Entity **CARTITEM** is a junction table representing the many-to-many relationship between CART and PRODUCTITEM. Each entry is identified by **CartItemID** and contains **Quantity** (the number of units chosen for each specific variant).

ORDER Entity The **ORDER** entity records finalized purchases. Each order is uniquely identified by an **OrderID** and linked to a CUSTOMER via **CustomerID**. Transaction attributes include **OrderDate**, **ShippingAddress**, and **PaymentMethod** (COD, Banking, Momo, ZaloPay). The system computes **TotalAmount** automatically based on selected variants and applied promotions. Each order references exactly one **SHIPPING** through **ShippingID** and may optionally apply a **VOUCHER** through **VoucherID**. The **Status** attribute tracks fulfillment progress ('Processing', 'Shipped', 'Delivered', 'Cancelled'). **DeliveredDate** records when the order was completed.

ORDERITEM Entity **ORDERITEM** is a weak entity representing individual product lines within an order. It is uniquely identified by composite key (**OrderID**, **ItemID**) and further references **ShopID** to determine the responsible seller. Each item records **Quantity** and **PriceAtPurchase** to preserve historical pricing data regardless of later modifications.

VOUCHER Entity The **VOUCHER** entity regulates promotional campaigns. Each voucher is characterized by **VoucherID**, **Code** (unique voucher code), **DiscountType** (Percentage or Amount), **DiscountValue** (discount amount/percentage), **MinOrderValue** (minimum order value to apply), **StartDate**, and **ExpiredDate**. **UsageLimit** controls maximum uses, and **UsedCount** tracks actual uses. **Status** indicates if the voucher is 'Active' or 'Expired'. Vouchers can be applied to orders subject to constraint checks.

SHIPPING Entity The **SHIPPING** entity defines available delivery options. Each shipping method is identified by **ShippingID** and described by **Name**, **EstimatedDays**, and **Fee**. Orders rely on this entity to determine logistics partners. **Status** can be 'Active' or 'Inactive'.

REVIEW Entity The feedback system incorporates a unified **REVIEW** entity to handle both product and shop reviews. A REVIEW is identified by **ReviewID** and associated with **CustomerID** and **TargetType** (either 'Product' or 'Shop'). **TargetID** indicates which product or shop is being reviewed. It stores evaluation details such as **Rating** (1-5), **Comment**, optional **ImageURL**, and **ReviewDate**. Only customers who have previously purchased from the seller (verified purchase) may submit such reviews.

3 Description of Semantic Constraints

3.1 ACCOUNT

- **Email** and **Password** fields are mandatory for all accounts, as they are required for authentication.
- The **Role** field can only accept one of the following values: Customer, Shop, or Admin, ensuring clear role-based access control.
- The **Status** field can only accept 'Active' or 'Ban'. A banned account cannot perform transactions or access certain features.
- **Phone** must be exactly 10 digits following Vietnamese phone format.

3.2 CUSTOMER

- **CustomerCode** is auto-generated with 'CUST' prefix (CUST00001, CUST00002, ...).
- A customer can only leave **Review** for items or shops they have actually purchased from, enforcing a verified purchase policy.

3.3 SHOP

- **ShopCode** is auto-generated with 'SHOP' prefix (SHOP00001, SHOP00002, ...).
- **Rating** field must always be within the range 0 to 5, reflecting customer feedback.
- **ShopStatus** can only accept 'Open', 'Temporarily Close', or 'Closed'.

3.4 ADMIN

- The **Role** field must specify the admin's level of access: Core, Moderator, or Support.

3.5 CATEGORY

- **CategoryName** field is mandatory and must not exceed 100 characters.
- **ParentCategoryID** can be null for root categories; otherwise enforces hierarchical organization.

3.6 PRODUCT

- **ProductName** and **ShopID** fields are mandatory for proper identification and ownership.
- **Status** can only be 'In stock' or 'Out of stock', automatically updated by triggers when stock reaches zero.

3.7 PRODUCTITEM

- **Price** must be ≥ 0, and **Stock** must be ≥ 0 to prevent invalid transactions.

3.8 CART

- **Quantity** must be ≥ 1 for each cart item.

3.9 ORDER

- **TotalAmount** cannot be negative, ensuring correct financial records.
- **Status** can only accept: Processing, Shipped, Delivered, or Cancelled.
- **DeliveredDate** must be $\geq$ **OrderDate** when set, ensuring chronological consistency.

3.10 ORDERITEM

- **Quantity** must be ≥ 1 to reflect the actual purchase.
- **PriceAtPurchase** must be ≥ 0, capturing the correct price at transaction time..

3.11 VOUCHER

- **DiscountValue** field must be ≥ 0.
- **DiscountType** field allows only 'Percentage' or 'Amount'.
- **MinOrderValue** and **UsageLimit** must be ≥ 0 and ≥ 1 respectively.
- **ExpiredDate** must be $\geq$ **StartDate**.

3.12 SHIPPING

- **EstimatedDays** must be ≥ 0, ensuring logical delivery time estimation.
- **Fee** must be ≥ 0.

3.13 REVIEW

- **Rating** field is mandatory and must be between 1 and 5.
- **TargetID** combined with **TargetType** uniquely identifies the reviewed entity.

4 Entity Relationship Diagram (ERD)

This is the Entity Relationship Diagram: HERE

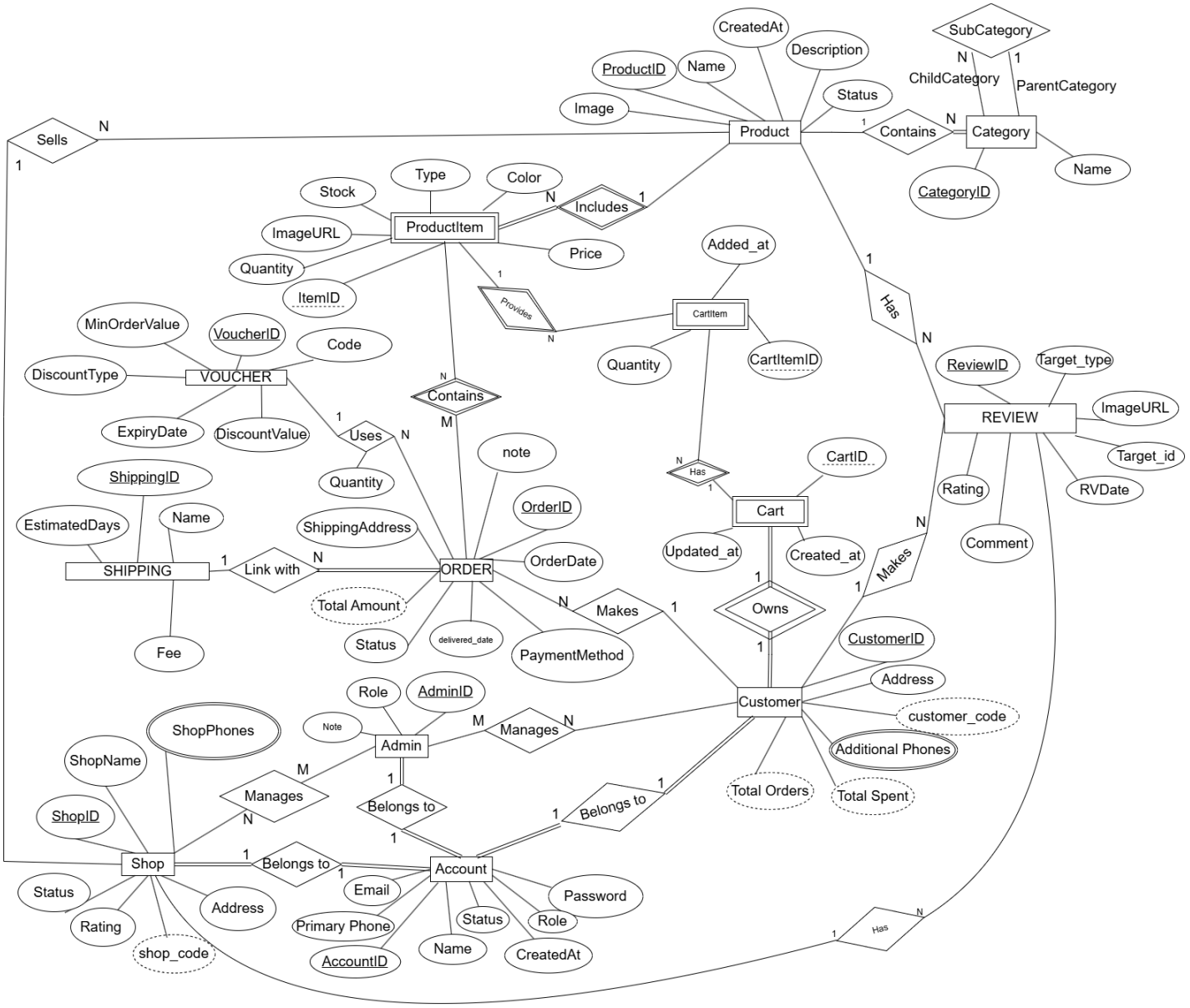

Figure 3: ERD diagram

5 Relational Database Schema

5.1 Mapping Diagram

This is the Relational Database Schema Diagram: HERE

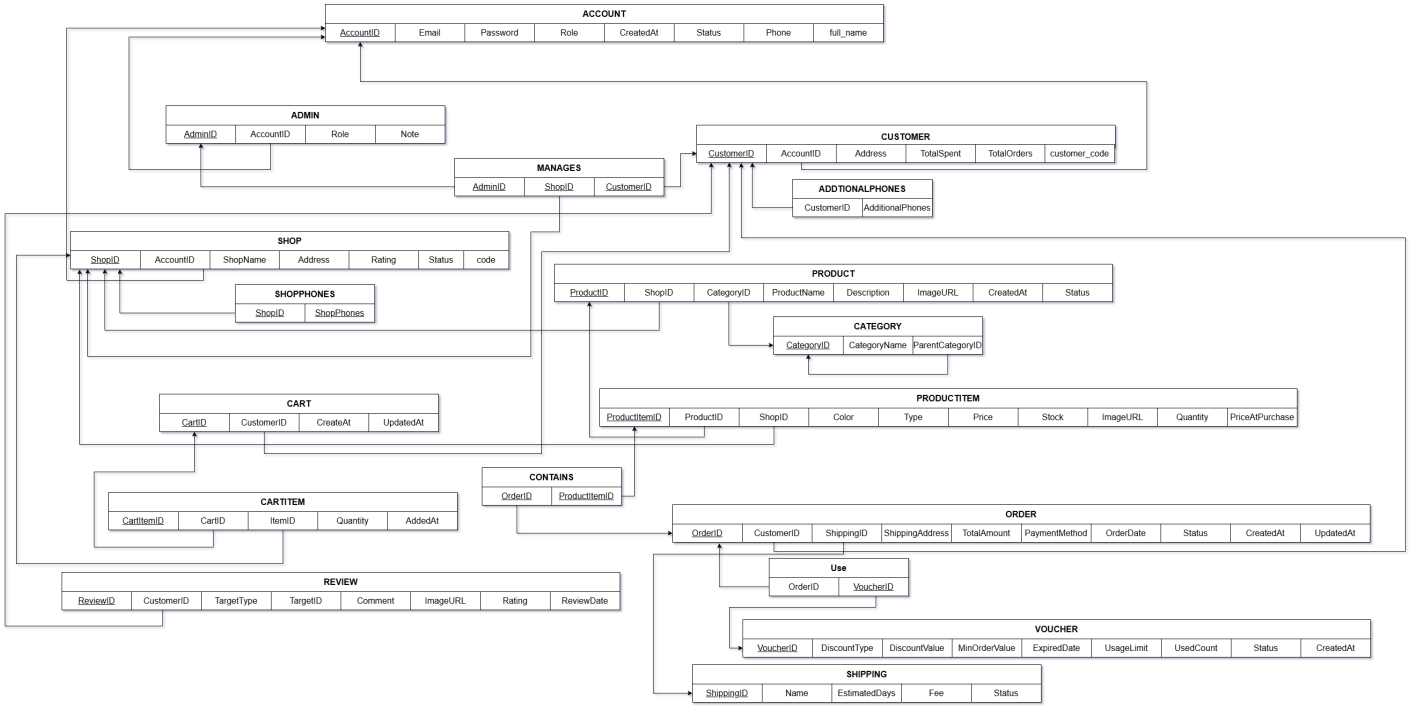

Figure 4: Relational Schema Diagram

5.2 Relational Database Schema Description

Table 1: Mapping to Database Schema

Name	Primary Key	Foreign Key
ACCOUNT	account_id	–
CUSTOMER	customer_id	customer_id → ACCOUNT.account_id
SHOP	shop_id	shop_id → ACCOUNT.account_id
ADMIN	admin_id	admin_id → ACCOUNT.account_id
CATEGORY	category_id	parent_category_id → CATEGORY.category_id
PRODUCT	product_id	shop_id → SHOP.shop_id; category_id → CATEGORY.category_id
PRODUCT-ITEM	item_id	product_id → PRODUCT.product_id; shop_id → SHOP.shop_id
CART	cart_id	customer_id → CUSTOMER.customer_id
CARTITEM	cart_item_id	cart_id → CART.cart_id; item_id → PRODUCTITEM.item_id
SHIPPING	shipping_id	–
VOUCHER	voucher_id	–
ORDER	order_id	customer_id → CUSTOMER.customer_id; shipping_id → SHIPPING.shipping_id; voucher_id → VOUCHER.voucher_id

Continued on next page

Table 1 – *Continued from previous page*

Name	Primary Key	Foreign Key
ORDERITEM	order_item_id	order_id → ORDER.order_id; item_id → PRODUCTITEM.item_id; shop_id → SHOP.shop_id
REVIEW	review_id	customer_id → CUSTOMER.customer_id

6 Selected Technology

6.1 Technology Stack

The proposed e-commerce platform utilizes a modern, scalable technology stack designed to support concurrent users, complex transactions, and real-time data synchronization:

6.1.1 Database Management System

MySQL Server 8.0 is selected as the primary DBMS due to its:

- **ACID Compliance**: Ensures data integrity across multi-seller transactions
- **Foreign Key Constraints**: Enforces referential integrity across 14 entities
- **Triggers and Stored Procedures**: Enables business logic automation (auto-generated codes, inventory management, rating calculations)
- **Recursive Queries**: Supports hierarchical category traversal
- **JSON Support**: Optional future extensibility for product metadata

6.1.2 Backend Framework

Node.js with Express.js provides:

- Event-driven, non-blocking I/O for handling multiple concurrent requests
- RESTful API design for clean separation between frontend and backend
- Middleware support for authentication, validation, and error handling
- Connection pooling for efficient database access

6.1.3 Frontend Framework

React with Vite offers:

- Component-based architecture for reusable UI elements
- State management via Redux for complex application state
- Fast hot module replacement during development
- Optimized production builds with tree-shaking

6.1.4 Authentication

JWT (JSON Web Tokens) for stateless authentication:

- Secure token-based sessions
- Role-based access control (Customer, Shop, Admin)
- Token expiration and refresh mechanisms

7 Database Implementation

7.1 Account Management

7.1.1 ACCOUNT Table

The **ACCOUNT** table serves as the central authentication and identity store for all system users. It implements table-level inheritance, with specialized entities (CUSTOMER, SHOP, ADMIN) storing additional role-specific information.

Listing 1: ACCOUNT Table Creation

```
CREATE TABLE Account (
    account_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role ENUM('Customer','Shop','Admin') NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    phone VARCHAR(20) NOT NULL CHECK (LENGTH(phone) = 10 AND phone REGEXP
        '^[0-9]{10}$'),
    status ENUM('Active','Ban') DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT CK_AccountRole CHECK (role IN ('Customer','Shop','Admin'))
);
```

General Description: The ACCOUNT table centralizes user authentication across all three user types. The **email** field is UNIQUE to prevent duplicate account registrations. The **phone** CHECK constraint enforces Vietnamese phone format validation at the database level. The **role** ENUM field determines user permissions (Customer can purchase, Shop can sell, Admin supervises). Status field enables account suspension without deletion.

7.1.2 CUSTOMER Table

Listing 2: CUSTOMER Table Creation

```
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    customer_code VARCHAR(20) UNIQUE,
```

```
    address VARCHAR(255),
    add_phone VARCHAR(20) CHECK (LENGTH(add_phone) = 10 AND add_phone REGEXP
        '^[0-9]{10}$'),
    total_spent DECIMAL(15,2) DEFAULT 0,
    total_order INT DEFAULT 0,
    FOREIGN KEY (customer_id) REFERENCES Account(account_id) ON DELETE CASCADE
);
```

General Description: The CUSTOMER table extends ACCOUNT with purchase-specific attributes. The **customer_code** is auto-generated via trigger with 'CUST' prefix, enabling easy customer identification in reports. **Address** is mandatory for order fulfillment. **TotalSpent** and **TotalOrder** are derived attributes maintained by triggers on order completion, supporting customer loyalty analysis. The UNIQUE constraint on customer_code and index on total_spent enable efficient customer ranking and loyalty program implementation.

7.1.3 SHOP Table

Listing 3: SHOP Table Creation

```
CREATE TABLE Shop (
    shop_id INT PRIMARY KEY,
    shop_code VARCHAR(20) UNIQUE,
    shop_name VARCHAR(255) NOT NULL,
    shop_phone VARCHAR(20) CHECK (LENGTH(shop_phone) = 10 AND shop_phone REGEXP
        '^[0-9]{10}$'),
    address_shop VARCHAR(255),
    rating DECIMAL(3,2) DEFAULT 0 CHECK (rating BETWEEN 0 AND 5),
    shop_status ENUM('Open','Temporarily Close','Closed') DEFAULT 'Open',
    FOREIGN KEY (shop_id) REFERENCES Account(account_id) ON DELETE CASCADE
);
```

General Description: The SHOP table models seller entities with business-specific attributes. **shop_code** is auto-generated with 'SHOP' prefix for seller identification. **shop_status** controls whether a seller can list products (Open) or is inactive. **rating** is auto-calculated from customer reviews via trigger, providing trust metrics. Indexes on shop_code and rating enable efficient shop searching and ranking by customer ratings.

7.1.4 ADMIN Table

Listing 4: ADMIN Table Creation

```
CREATE TABLE Admin (
    admin_id INT PRIMARY KEY,
    role ENUM('Core','Moderator','Support') NOT NULL,
    note TEXT,
    FOREIGN KEY (admin_id) REFERENCES Account(account_id) ON DELETE CASCADE
```

```
);
```

General Description: The ADMIN table represents platform supervisors with three access levels. **Core** admins have full system access, **Moderator** admins handle dispute resolution and content moderation, while **Support** admins assist customers. The note field allows tracking of admin activities and responsibilities. Unlike CUSTOMER and SHOP, ADMIN entities do not engage in transactions.

7.2 Product Catalog

7.2.1 CATEGORY Table

Listing 5: CATEGORY Table Creation

```
CREATE TABLE Category (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL,
    parent_category_id INT,
    FOREIGN KEY (parent_category_id) REFERENCES Category(category_id) ON DELETE SET
        NULL
);
```

General Description: The CATEGORY table implements a self-referential recursive relationship enabling hierarchical product organization (e.g., Electronics → Phones → Smartphones). The **parent_category_id** can be NULL for root-level categories. The recursive FK with ON DELETE SET NULL ensures child categories remain accessible if a parent is deleted..

7.2.2 PRODUCT Table

Listing 6: PRODUCT Table Creation

```
CREATE TABLE Product (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    shop_id INT NOT NULL,
    category_id INT NOT NULL,
    product_name VARCHAR(255) NOT NULL,
    description TEXT,
    image VARCHAR(255),
    rating DECIMAL(3,2) DEFAULT 0,
    status ENUM('In stock','Out of stock') DEFAULT 'In stock',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (shop_id) REFERENCES Shop(shop_id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES Category(category_id)
);
```

General Description: The PRODUCT table represents individual product listings managed by shops. Each product belongs to exactly one SHOP (N:1 relationship with ON DELETE CASCADE to remove products when shop is deleted) and one CATEGORY (N:1 with ON DELETE RESTRICT to preserve category structure). The **rating** is auto-calculated from customer reviews via trigger. **status** tracks inventory availability and is auto-updated by triggers when stock reaches zero. The updated_at timestamp enables tracking product modification history.

7.2.3 PRODUCTITEM Table

Listing 7: PRODUCTITEM Table Creation

```
CREATE TABLE ProductItem (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT NOT NULL,
    shop_id INT NOT NULL,
    color VARCHAR(50) NOT NULL,
    type VARCHAR(50) NOT NULL,
    price DECIMAL(15,2) NOT NULL CHECK(price >= 0),
    stock INT NOT NULL CHECK(stock >= 0),
    image_url VARCHAR(255),
    FOREIGN KEY (product_id) REFERENCES Product(product_id) ON DELETE CASCADE,
    FOREIGN KEY (shop_id) REFERENCES Shop(shop_id) ON DELETE CASCADE
);
```

General Description: The PRODUCTITEM (Variant) table represents purchasable variations of products. The UNIQUE constraint on (product_id, color, type) prevents duplicate variants of the same product. The **stock** field tracks inventory levels and is decremented by triggers on order placement, incremented on order cancellation. The **price** field is stored for each variant, enabling different pricing for sizes/colors.

7.3 Shopping Cart

7.3.1 CART Table

Listing 8: CART Table Creation

```
CREATE TABLE Cart (
    cart_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id) ON DELETE CASCADE
);
```

General Description: The CART table implements a 1:1 relationship with CUSTOMER, ensuring each customer has exactly one shopping cart. The UNIQUE constraint on customer_id enforces this one-to-one relationship. The updated_at timestamp tracks cart modification time, useful for session management. ON DELETE CASCADE ensures cart deletion when customer account is removed.

7.3.2 CARTITEM Table

Listing 9: CARTITEM Table Creation

```
CREATE TABLE CartItem (
    cart_item_id INT AUTO_INCREMENT PRIMARY KEY,
    cart_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT DEFAULT 1 CHECK(quantity >= 1),
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (cart_id) REFERENCES Cart(cart_id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES ProductItem(item_id) ON DELETE CASCADE,
    UNIQUE KEY unique_cart_item (cart_id, item_id)
);
```

General Description: The CARTITEM table implements the N:M relationship between CART and PRODUCTITEM. The UNIQUE constraint on (cart_id, item_id) prevents duplicate items in the same cart. The **quantity** field tracks the number of units selected, with CHECK constraint ensuring at least 1 unit. ON DELETE CASCADE ensures cart items are removed when either cart or product is deleted.

7.4 Orders and Items

7.4.1 ORDER Table

Listing 10: ORDER Table Creation

```
CREATE TABLE `Order` (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    shipping_id INT NOT NULL,
    voucher_id INT,
    status ENUM('Processing','Shipped','Delivered','Cancelled') DEFAULT
        'Processing',
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    delivered_date TIMESTAMP NULL,
    shipping_address VARCHAR(255) NOT NULL,
    total_amount DECIMAL(15,2) DEFAULT 0,
    payment_method ENUM('COD','Banking','Momo','ZaloPay') NOT NULL DEFAULT 'COD',
    note TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (shipping_id) REFERENCES Shipping(shipping_id),
    FOREIGN KEY (voucher_id) REFERENCES Voucher(voucher_id) ON DELETE SET NULL,
    CONSTRAINT CK_OrderDateRange CHECK (delivered_date IS NULL OR delivered_date >=
        order_date)
);
```

General Description: The ORDER table records all purchase transactions. The **status** ENUM tracks order lifecycle (Processing → Shipped → Delivered or Cancelled). The **voucher_id** is nullable (N:0..1 relationship) since orders may not use vouchers. The CHECK constraint `delivered_date >= order_date` enforces chronological consistency. Indexes on customer_id, status, and order_date enable efficient order retrieval and reporting. ON DELETE RESTRICT on shipping_id prevents accidental deletion of shipping methods in use.

7.4.2 ORDERITEM Table

Listing 11: ORDERITEM Table Creation

```
CREATE TABLE OrderItem (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    shop_id INT NOT NULL,
    quantity INT NOT NULL CHECK(quantity >= 1),
    price_at_purchase DECIMAL(15,2) NOT NULL CHECK(price_at_purchase >= 0),
    FOREIGN KEY (order_id) REFERENCES `Order`(order_id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES ProductItem(item_id),
    FOREIGN KEY (shop_id) REFERENCES Shop(shop_id)
);
```

General Description: The ORDERITEM table represents individual line items within an order, supporting multi-seller purchases. The **price_at_purchase** field captures the price at transaction time, preserving historical accuracy even if product prices change. The **shop_id** field identifies the seller responsible for fulfilling each line item. The UNIQUE constraint on (order_id, item_id) prevents duplicate line items. ON DELETE RESTRICT on item_id and shop_id ensures order history remains intact.

7.5 Promotions

7.5.1 VOUCHER Table

Listing 12: VOUCHER Table Creation

```
CREATE TABLE Voucher (
    voucher_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    code VARCHAR(50) UNIQUE NOT NULL,
    discount_type ENUM('Percentage','Amount') NOT NULL,
    discount_value DECIMAL(10,2) NOT NULL CHECK(discount_value >= 0),
    min_order_value DECIMAL(15,2) DEFAULT 0 CHECK(min_order_value >= 0),
    start_date DATE NOT NULL,
    expired_date DATE NOT NULL,
    usage_limit INT DEFAULT 1 CHECK(usage_limit >= 1),
    used_count INT DEFAULT 0,
    status ENUM('Active','Expired') DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT CK_VoucherDateRange CHECK (expired_date >= start_date)
);
```

General Description: The VOUCHER table manages promotional campaigns with flexible discount mechanisms. The **discount_type** ENUM supports both percentage (e.g., 10%) and fixed amount (e.g., 50,000 VND) discounts. The **min_order_value** enforces minimum purchase requirements. The **usage_limit** and **used_count** fields track voucher usage, preventing over-utilization. The CHECK constraint ensures used_count never exceeds usage_limit.

7.6 Shipping

7.6.1 SHIPPING Table

Listing 13: SHIPPING Table Creation

```
CREATE TABLE Shipping (
    shipping_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    estimated_days INT NOT NULL CHECK(estimated_days >= 0),
    fee DECIMAL(10,2) NOT NULL CHECK(fee >= 0),
    status ENUM('Active','Inactive') DEFAULT 'Active'
);
```

General Description: The SHIPPING table defines available delivery options for orders. Each record represents a logistics partner or delivery method. The **estimated_days** field provides expected delivery timeframes. The **fee** field enables dynamic shipping cost calculation. The **status** field controls availability (Active methods appear in checkout, Inactive ones are hidden).

7.7 Reviews and Feedback

7.7.1 REVIEW Table

Listing 14: REVIEW Table Creation

```
CREATE TABLE Review (
    review_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    customer_id INT NOT NULL,
    target_type ENUM('Shop','Product') NOT NULL,
    target_id INT NOT NULL,
    rating INT NOT NULL CHECK(rating BETWEEN 1 AND 5),
    comment TEXT NOT NULL,
    image_url VARCHAR(255),
    review_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id) ON DELETE CASCADE
);
```

8 Sample Data Insertion

This section demonstrates sample data for testing the database schema. Data is organized by entity relationships to maintain referential integrity.

8.1 Account Data

Listing 15: Sample ACCOUNT Insert

```
-- Insert 7 accounts: 3 customers, 3 shops, 1 admin
INSERT INTO Account (email, password, role, full_name, phone, status)
VALUES
    ('c1@gmail.com', '1234', 'Customer', 'Nguyen Van A', '0901111111', 'Active'),
    ('c2@gmail.com', '1234', 'Customer', 'Tran Thi B', '0902222222', 'Active'),
    ('c3@gmail.com', '1234', 'Customer', 'Le Hoai C', '0903333333', 'Active'),
    ('s1@gmail.com', '1234', 'Shop', 'Shop ABC', '0904444444', 'Active'),
    ('s2@gmail.com', '1234', 'Shop', 'Shop XYZ', '0905555555', 'Active'),
    ('s3@gmail.com', '1234', 'Shop', 'Tech Store', '0906666666', 'Active'),
    ('admin1@gmail.com', '1234', 'Admin', 'Admin Core', '0907777777', 'Active');
```

8.2 Customer Data

Listing 16: Sample CUSTOMER Insert

```
-- Triggers automatically create CUSTOMER records during INSERT
-- This example shows updating customer info with addresses
UPDATE Customer SET
    address = '123 Nguyen Hue, District 1, HCMC',
    add_phone = '0908888888'
WHERE customer_id = 1;

UPDATE Customer SET
    address = '456 Le Loi, District 3, HCMC',
    add_phone = '0909999999'
WHERE customer_id = 2;

```

```sql
UPDATE Customer SET
    address = '789 Tran Hung Dao, Hoan Kiem, Hanoi',
    add_phone = '0911111111'
WHERE customer_id = 3;
```

8.3 Shop Data

Listing 17: Sample SHOP Insert

```sql
-- Triggers automatically create SHOP records
-- Update shop information
UPDATE Shop SET
    shop_name = 'Shop ABC - Electronics',
    shop_phone = '0904444444',
    address_shop = '12 Vo Van Kiet, District 1',
    rating = 4.50,
    shop_status = 'Open'
WHERE shop_id = 4;

UPDATE Shop SET
    shop_name = 'Shop XYZ - Fashion',
    shop_phone = '0905555555',
    address_shop = '45 Hai Ba Trung, District 3',
    rating = 0,
    shop_status = 'Open'
WHERE shop_id = 5;

UPDATE Shop SET
    shop_name = 'Tech Store - Premium',
    shop_phone = '0906666666',
    address_shop = '67 Nguyen Dinh Chieu, District 7',
    rating = 5.00,
    shop_status = 'Open'
WHERE shop_id = 6;
```

8.4 Category Data

Listing 18: Sample CATEGORY Insert

```sql
INSERT INTO Category (category_name, parent_category_id)
VALUES
    ('Electronics', NULL),          -- Root: ID 1
    ('Fashion', NULL),              -- Root: ID 2
    ('Phones', 1),                  -- Child of Electronics
    ('Laptops', 1),                 -- Child of Electronics
    ('Clothes', 2),                 -- Child of Fashion
    ('Shoes', 2);                   -- Child of Fashion
```

8.5 Product and ProductItem Data

Listing 19: Sample PRODUCT Insert

```sql
-- Insert products from different shops
INSERT INTO Product (shop_id, category_id, product_name,
                     description, image, status)
VALUES
    (4, 3, 'iPhone 15 Pro',
     'Latest Apple flagship with A17 Pro chip',
     'iphone15pro.jpg', 'In stock'),

    (4, 3, 'Samsung Galaxy S24',
     'Latest Samsung with AI features',
     'galaxyS24.jpg', 'In stock'),

    (5, 5, 'Premium Cotton T-Shirt',
     'High-quality 100% cotton shirt',
     'tshirt.jpg', 'In stock'),

    (6, 4, 'MacBook Air M3',
     'Lightweight laptop for professionals',
     'macbookm3.jpg', 'In stock'),

    (6, 3, 'Sony Headset XM5',
     'Noise-cancelling wireless headphones',
     'headset.jpg', 'In stock');
```

Listing 20: Sample PRODUCTITEM Insert

```sql
-- Insert product variants (color, size, capacity)
INSERT INTO ProductItem (product_id, shop_id, color, type,
                         price, stock, image_url)
VALUES
    -- iPhone 15 Pro variants (Product 1, Shop 4)
    (1, 4, 'Space Black', '128GB', 1099000, 50,
     'iphone15pro-black-128.jpg'),

    (1, 4, 'Blue Titanium', '256GB', 1199000, 30,
     'iphone15pro-blue-256.jpg'),

    -- Samsung Galaxy S24 variants (Product 2, Shop 4)
    (2, 4, 'Phantom Black', '256GB', 920000, 40,
     'galaxyS24-black-256.jpg'),

    (2, 4, 'Amber Gold', '512GB', 1099000, 25,
     'galaxyS24-gold-512.jpg'),

    -- T-Shirt variants (Product 3, Shop 5)
    (3, 5, 'White', 'Size M', 150000, 100,
```

```
    'tshirt-white-m.jpg'),

    (3, 5, 'Black', 'Size L', 150000, 80,
    'tshirt-black-l.jpg'),

    -- MacBook Air M3 variants (Product 4, Shop 6)
    (4, 6, 'Silver', '16GB RAM', 2299000, 15,
    'macbookm3-silver-16gb.jpg'),

    (4, 6, 'Space Gray', '24GB RAM', 2599000, 10,
    'macbookm3-gray-24gb.jpg'),

    -- Sony Headset variants (Product 5, Shop 6)
    (5, 6, 'Black', 'Standard', 4990000, 60,
    'sonyxm5-black.jpg'),

    (5, 6, 'Silver', 'Standard', 4990000, 40,
    'sonyxm5-silver.jpg');
```

8.6 Shipping and Voucher Data

Listing 21: Sample SHIPPING Insert

```
INSERT INTO Shipping (name, estimated_days, fee, status)
VALUES
    ('GHN Express', 2, 15000, 'Active'),
    ('J&T Standard', 3, 12000, 'Active'),
    ('Shopee Express', 1, 20000, 'Active'),
    ('Ninja Van Economy', 4, 9000, 'Active');
```

Listing 22: Sample VOUCHER Insert

```
INSERT INTO Voucher (code, discount_type, discount_value,
                     min_order_value, start_date, expired_date,
                     usage_limit, status)
VALUES
    -- 10% discount, min 500K
    ('DISCOUNT10', 'Percentage', 10, 500000,
    '2025-01-01', '2025-12-31', 1000, 'Active'),

    -- Fixed 100K discount, min 1M
    ('SAVE100K', 'Amount', 100000, 1000000,
    '2025-06-01', '2025-12-31', 500, 'Active'),

    -- New Year promotion: 5% off
    ('NEWYEAR2025', 'Percentage', 5, 200000,
    '2025-12-01', '2026-01-31', 2000, 'Active'),

    -- Expired voucher (for testing)
```

```
    ('OLDCODE', 'Percentage', 15, 300000,
     '2024-01-01', '2024-12-31', 100, 'Expired');
```

8.7 Order Transaction Data

Listing 23: Sample ORDER Insert

```
-- Note: Cart and CartItem are auto-created by triggers
-- Orders created from shopping carts

-- Order 1: Customer 1, 2 items from Shop 4
INSERT INTO \`Order\` (customer_id, shipping_id, voucher_id, status,
                       shipping_address, total_amount,
                       payment_method, note)
VALUES
    (1, 1, 1, 'Delivered',
     '123 Nguyen Hue, District 1, HCMC', 2098000,
     'Banking', 'Received in good condition');

-- Insert order items for Order 1
INSERT INTO OrderItem (order_id, item_id, shop_id, quantity,
                       price_at_purchase)
VALUES
    (1, 1, 4, 1, 1099000), -- iPhone 15 Pro Black 128GB
    (1, 2, 4, 1, 1199000); -- iPhone 15 Pro Blue 256GB

-- Order 2: Customer 2, cancelled order
INSERT INTO \`Order\` (customer_id, shipping_id, voucher_id, status,
                       shipping_address, total_amount,
                       payment_method, note)
VALUES
    (2, 2, NULL, 'Cancelled',
     '456 Le Loi, District 3, HCMC', 932000,
     'COD', 'Cancelled by customer');

INSERT INTO OrderItem (order_id, item_id, shop_id, quantity,
                       price_at_purchase)
VALUES
    (2, 3, 4, 1, 920000); -- Galaxy S24 Black 256GB

-- Order 3: Customer 3, delivered
INSERT INTO \`Order\` (customer_id, shipping_id, voucher_id, status,
                       shipping_address, total_amount,
                       payment_method, note)
VALUES
    (3, 3, 2, 'Delivered',
     '789 Tran Hung Dao, Hanoi', 170000,
     'Momo', 'Please leave at reception');

```

```sql
INSERT INTO OrderItem (order_id, item_id, shop_id, quantity,
                       price_at_purchase)
VALUES
    (3, 5, 5, 1, 150000); -- T-Shirt White Size M

-- Order 4: Customer 1, shipped
INSERT INTO \‘Order\‘ (customer_id, shipping_id, voucher_id, status,
                      shipping_address, total_amount,
                      payment_method, note)
VALUES
    (1, 4, NULL, ’Shipped’,
     ’123 Nguyen Hue, District 1, HCMC’, 5009000,
     ’ZaloPay’, NULL);

INSERT INTO OrderItem (order_id, item_id, shop_id, quantity,
                       price_at_purchase)
VALUES
    (4, 9, 6, 1, 4990000); -- Sony Headset Black
```

8.8 Review Data

Listing 24: Sample REVIEW Insert

```sql
-- Reviews only from verified purchases (Delivered orders)

-- Review 1: Customer 1 reviewed Product 1 (iPhone 15)
INSERT INTO Review (customer_id, target_type, target_id,
                    rating, comment, image_url)
VALUES
    (1, ’Product’, 1, 5,
     ’Excellent phone! Fast performance and great camera’,
     ’review_iphone.jpg’),

    -- Review 2: Customer 1 reviewed Shop 4
    (1, ’Shop’, 4, 5,
     ’Very reliable seller, fast shipping’,
     NULL),

    -- Review 3: Customer 3 reviewed Product 3 (T-Shirt)
    (3, ’Product’, 3, 4,
     ’Good quality cotton, true to size’,
     ’review_tshirt.jpg’),

    -- Review 4: Customer 3 reviewed Shop 5
    (3, ’Shop’, 5, 4,
     ’Decent shop, packaging could be better’,
     NULL),

    -- Review 5: Customer 1 reviewed Product 5 (Headset)
```

```
    (1, 'Product', 5, 5,
     'Amazing sound quality, very comfortable',
     'review_headset.jpg'),

    -- Review 6: Customer 1 reviewed Shop 6
    (1, 'Shop', 6, 5,
     'Premium products, excellent customer service',
     NULL);
```

9 Functions and Procedures

This section details the business logic automation implemented through MySQL stored functions, procedures, and triggers.

9.1 Account Management Functions

9.1.1 Function: Calculate Customer Total Spent

Purpose: Calculates the total amount spent by a customer across all non-cancelled orders, including shipping fees and voucher discounts.

Listing 25: Function: fn_get_customer_total_spent

```
CREATE FUNCTION fn_get_customer_total_spent(p_customer_id INT)
RETURNS DECIMAL(15, 2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_total_spent DECIMAL(15, 2);

    SELECT COALESCE(SUM(fn_calculate_order_total(o.order_id)), 0)
    INTO v_total_spent
    FROM `Order` o
    WHERE o.customer_id = p_customer_id
      AND o.status != 'Cancelled';

    RETURN v_total_spent;
END$$
```

Related Tables: ACCOUNT, CUSTOMER, ORDER, ORDERITEM, SHIPPING, VOUCHER

Explanation: This function sums all completed order totals for a specific customer. It queries the ORDER table filtered by customer_id and status != 'Cancelled', then aggregates order totals including shipping fees and subtracting applied voucher discounts. The function is DETERMINISTIC (same input always returns same output) and reads SQL data only.

Example Usage:

```
-- Get total spent by customer ID 1
SELECT fn_get_customer_total_spent(1);
-- Result: 2,069,500.00 VND
-- Customer 1 orders: Order 1 (1,985,000) + Order 4 (84,500)

-- Get total spent by customer ID 2
SELECT fn_get_customer_total_spent(2);
-- Result: 0.00 VND
-- Customer 2 only has cancelled orders

-- Get total spent by customer ID 3
SELECT fn_get_customer_total_spent(3);
-- Result: 31,000.00 VND
-- Customer 3: Order 3 (31,000)

-- Get customer info with total spent
SELECT c.customer_code, a.full_name,
       fn_get_customer_total_spent(c.customer_id) AS total_spent
FROM Customer c
INNER JOIN Account a ON c.customer_id = a.account_id
WHERE c.customer_id = 1;
```

9.2 Shop Management Functions

9.2.1 Function: Calculate Shop Revenue

Purpose: Calculates total revenue generated by a shop from all non-cancelled orders, accounting for multi-seller scenarios.

Listing 26: Function: fn_get_shop_revenue

```
CREATE FUNCTION fn_get_shop_revenue(p_shop_id INT)
RETURNS DECIMAL(15, 2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_revenue DECIMAL(15, 2);
    DECLARE v_subtotal DECIMAL(15, 2);
    DECLARE v_discount_total DECIMAL(15, 2);

    -- Calculate shop revenue (OrderItem + Shipping - Voucher discount)
    -- Only count non-cancelled orders

    -- Calculate subtotal from OrderItem
    SELECT COALESCE(SUM(oi.quantity * oi.price_at_purchase), 0)
    INTO v_subtotal
    FROM OrderItem oi
    INNER JOIN `Order` o ON oi.order_id = o.order_id
```

```
    WHERE oi.shop_id = p_shop_id AND o.status != 'Cancelled';

    -- Calculate voucher discount (only if min_order_value met)
    SELECT COALESCE(SUM(
        CASE
            WHEN v.discount_type = 'Percentage' AND v_subtotal >=
                COALESCE(v.min_order_value, 0)
                 THEN v_subtotal * v.discount_value / 100
            WHEN v.discount_type = 'Amount' AND v_subtotal >=
                COALESCE(v.min_order_value, 0)
                 THEN v.discount_value
            ELSE 0
        END
    ), 0)
    INTO v_discount_total
    FROM `Order` o
    LEFT JOIN Voucher v ON o.voucher_id = v.voucher_id
    WHERE o.order_id IN (
        SELECT DISTINCT oi.order_id
        FROM OrderItem oi
        WHERE oi.shop_id = p_shop_id
    )
    AND o.status != 'Cancelled';

    SET v_revenue = v_subtotal - v_discount_total;

    IF v_revenue < 0 THEN
        SET v_revenue = 0;
    END IF;

    RETURN v_revenue;
END$$
```

Related Tables: SHOP, ORDERITEM, ORDER

Explanation: This function calculates shop revenue by summing all OrderItem line items (quantity × price_at_purchase) where the shop_id matches and the order status is not 'Cancelled'. Since orders can contain items from multiple shops, using shop_id in OrderItem ensures accurate shop-specific revenue calculation.

Example Usage:

```
-- Get revenue for shop ID 4
SELECT fn_get_shop_revenue(4);
-- Result: 1,980,000.00 VND
-- Calculation: Shop 4 items (1,000,000 + 1,200,000) - voucher (220,000)

-- Get revenue for shop ID 5
SELECT fn_get_shop_revenue(5);
```

```
-- Result: 25,000.00 VND

-- Get revenue for shop ID 6
SELECT fn_get_shop_revenue(6);
-- Result: 80,000.00 VND

-- Top shops by revenue
SELECT s.shop_code, s.shop_name, fn_get_shop_revenue(s.shop_id) AS revenue
FROM Shop s
ORDER BY fn_get_shop_revenue(s.shop_id) DESC
LIMIT 5;
```

9.2.2 Function: Calculate Shop Rating

Purpose: Calculates the average rating of a shop based on verified customer reviews.

Listing 27: Function: fn_calculate_shop_rating

```
CREATE FUNCTION fn_calculate_shop_rating(p_shop_id INT)
RETURNS DECIMAL(3, 2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_avg_rating DECIMAL(3, 2);

    SELECT COALESCE(AVG(r.rating), 0)
    INTO v_avg_rating
    FROM Review r
    WHERE r.target_type = 'Shop'
      AND r.target_id = p_shop_id;

    RETURN v_avg_rating;
END$$
```

Related Tables: SHOP, REVIEW

Explanation: This function queries the REVIEW table where target_type='Shop' and target_id matches the shop_id, then calculates the average rating. If no reviews exist, it returns 0. The function is called by a trigger whenever a new review is added to keep the shop rating current.

Example Usage:

```
-- Get shop rating for shop 4
SELECT fn_calculate_shop_rating(4);
-- Result: 4.00
-- Reviews for Shop 4: (5 + 3) / 2 = 4.00

```

```sql
-- Get shop rating for shop 5
SELECT fn_calculate_shop_rating(5);
-- Result: 0.00
-- No reviews for Shop 5

-- Get shop rating for shop 6
SELECT fn_calculate_shop_rating(6);
-- Result: 5.00
-- Review for Shop 6: 5 stars

-- Get shop info with calculated rating
SELECT s.shop_code, s.shop_name, s.rating AS stored_rating,
       fn_calculate_shop_rating(s.shop_id) AS calculated_rating
FROM Shop s
WHERE s.shop_id = 4;
```

—

9.3 Product Management Functions

9.3.1 Function: Calculate Product Rating

Purpose: Calculates the average rating of a product based on verified customer reviews.

Listing 28: Function: fn_calculate_product_rating

```sql
CREATE FUNCTION fn_calculate_product_rating(p_product_id INT)
RETURNS DECIMAL(3, 2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_avg_rating DECIMAL(3, 2);

    SELECT COALESCE(AVG(r.rating), 0)
    INTO v_avg_rating
    FROM Review r
    WHERE r.target_type = 'Product'
      AND r.target_id = p_product_id;

    RETURN v_avg_rating;
END$$
```

Related Tables: PRODUCT, REVIEW

Explanation: Similar to shop rating function, this calculates product average rating from REVIEW table where target_type='Product'. Returns 0 if no reviews exist. Used by triggers to maintain denormalized rating in PRODUCT table for query performance.

Example Usage:

```sql
-- Get rating for product 1
SELECT fn_calculate_product_rating(1);
-- Result: 4.50
-- Reviews for Product 1: (5 + 4) / 2 = 4.50

-- Get rating for product 3
SELECT fn_calculate_product_rating(3);
-- Result: 4.00
-- Review for Product 3: 4 stars

-- Get rating for product 4
SELECT fn_calculate_product_rating(4);
-- Result: 5.00
-- Review for Product 4: 5 stars

-- Get top-rated products
SELECT p.product_id, p.product_name,
       fn_calculate_product_rating(p.product_id) AS rating,
       COUNT(r.review_id) AS review_count
FROM Product p
LEFT JOIN Review r ON p.product_id = r.target_id
                  AND r.target_type = 'Product'
GROUP BY p.product_id
ORDER BY rating DESC
LIMIT 10;
```

9.3.2 Function: Calculate Order Total

Purpose: Calculates the final order total including subtotal, shipping fee, and voucher discount with minimum order value validation.

Listing 29: Function: fn_calculate_order_total

```sql
CREATE FUNCTION fn_calculate_order_total(p_order_id INT)
RETURNS DECIMAL(15, 2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_subtotal DECIMAL(15, 2);
    DECLARE v_shipping_fee DECIMAL(10, 2);
    DECLARE v_discount DECIMAL(10, 2);
    DECLARE v_discount_type VARCHAR(20);
    DECLARE v_discount_value DECIMAL(10, 2);
    DECLARE v_min_order_value DECIMAL(10, 2);
    DECLARE v_total DECIMAL(15, 2);

    -- Calculate subtotal from order items
    SELECT COALESCE(SUM(oi.quantity * oi.price_at_purchase), 0)
    INTO v_subtotal
```

```
    FROM OrderItem oi
    WHERE oi.order_id = p_order_id;

    -- Get shipping fee
    SELECT COALESCE(sh.fee, 0)
    INTO v_shipping_fee
    FROM `Order` o
    LEFT JOIN Shipping sh ON o.shipping_id = sh.shipping_id
    WHERE o.order_id = p_order_id;

    -- Get voucher discount with min_order_value check
    SELECT v.discount_type, COALESCE(v.discount_value, 0),
        COALESCE(v.min_order_value, 0)
    INTO v_discount_type, v_discount_value, v_min_order_value
    FROM `Order` o
    LEFT JOIN Voucher v ON o.voucher_id = v.voucher_id
    WHERE o.order_id = p_order_id;

    -- Calculate discount amount (only if subtotal meets minimum requirement)
    SET v_discount = 0;
    IF v_discount_type IS NOT NULL AND v_subtotal >= v_min_order_value THEN
        IF v_discount_type = 'Percentage' THEN
            SET v_discount = v_subtotal * (v_discount_value / 100);
        ELSEIF v_discount_type = 'Amount' THEN
            SET v_discount = v_discount_value;
        END IF;
    END IF;

    -- Calculate total = subtotal + shipping - discount
    SET v_total = v_subtotal + v_shipping_fee - v_discount;

    RETURN v_total;
END$$
```

Related Tables: ORDER, ORDERITEM, SHIPPING, VOUCHER

Explanation: This comprehensive function calculates order totals by:

1. Summing all OrderItem line items (quantity × price_at_purchase)
2. Adding shipping fee from related Shipping record
3. Calculating voucher discount (percentage or fixed amount) with min_order_value validation
4. Subtracting discount from subtotal + shipping
5. Ensuring final total is non-negative

Example Usage:

```
-- Calculate order total for order 1
SELECT fn_calculate_order_total(1);
-- Result: 1,985,000.00 VND
-- Calculation: (1,000,000 + 1,200,000) + 5,000 - 220,000

-- Calculate order total for order 2
SELECT fn_calculate_order_total(2);
-- Result: 904,000.00 VND
-- Calculation: 900,000 + 4,000

-- Get order summary with calculated total
SELECT o.order_id, o.order_date, a.full_name,
       fn_calculate_order_total(o.order_id) AS total_amount,
       o.status, o.payment_method
FROM `Order` o
INNER JOIN Customer c ON o.customer_id = c.customer_id
INNER JOIN Account a ON c.customer_id = a.account_id
WHERE o.order_id = 1;
```

9.3.3 Function: Get Order Total Items with Cursor

Purpose: Demonstrates cursor usage to loop through OrderItems and calculate total quantity in an order.

Listing 30: Function: fn_get_order_total_items_with_cursor

```
CREATE FUNCTION fn_get_order_total_items_with_cursor(p_order_id INT)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_total_items INT DEFAULT 0;
    DECLARE v_done INT DEFAULT FALSE;
    DECLARE v_item_quantity INT;

    -- Declare cursor for OrderItems
    DECLARE order_items_cursor CURSOR FOR
        SELECT quantity
        FROM OrderItem
        WHERE order_id = p_order_id;

    -- Declare continue handler for cursor
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

    -- Validate order exists
    IF NOT EXISTS (SELECT 1 FROM `Order` WHERE order_id = p_order_id) THEN
        RETURN 0;
    END IF;

```

```
    -- Open cursor
    OPEN order_items_cursor;

    -- Loop through each item
    item_loop: LOOP
        FETCH order_items_cursor INTO v_item_quantity;

        IF v_done THEN
            LEAVE item_loop;
        END IF;

        SET v_total_items = v_total_items + v_item_quantity;
    END LOOP item_loop;

    -- Close cursor
    CLOSE order_items_cursor;

    RETURN v_total_items;
END$$
DELIMITER ;
```

Related Tables: ORDER, ORDERITEM

Explanation: This function demonstrates advanced cursor usage in MySQL stored functions. It:

1. Declares a cursor to iterate through all OrderItems for a specific order
2. Uses CONTINUE HANDLER to catch NOT FOUND exception when cursor reaches end
3. Loops through each item, accumulating total quantity
4. Returns cumulative total items count

Example Usage:

```
-- Get total items in order 1
SELECT fn_get_order_total_items_with_cursor(1);
-- Result: 2
-- Order 1 has 2 items (qty 1 + qty 1 = 2 total units)

-- Get total items in order 2
SELECT fn_get_order_total_items_with_cursor(2);
-- Result: 1
-- Order 2 has 1 item (qty 1)

-- Get total items for non-existent order
SELECT fn_get_order_total_items_with_cursor(999);
-- Result: 0
-- Order 999 does not exist
```

```
15
16 -- Order summary with total items
17 SELECT o.order_id,
18        COUNT(oi.order_item_id) AS line_items,
19        fn_get_order_total_items_with_cursor(o.order_id) AS total_items
20 FROM `Order` o
21 LEFT JOIN OrderItem oi ON o.order_id = oi.order_id
22 WHERE o.order_id = 1
23 GROUP BY o.order_id;
```

9.4 Category Hierarchy Procedures

9.4.1 Procedure: Get Category Hierarchy (All Levels)

Purpose: Displays complete hierarchical structure of all categories using recursive CTE (Common Table Expression).

Listing 31: Procedure: sp_get_category_hierarchy

```
1  CREATE PROCEDURE sp_get_category_hierarchy()
2  BEGIN
3      WITH RECURSIVE category_tree AS (
4          -- Anchor: Get all root categories (parent_category_id IS NULL)
5          SELECT
6              category_id,
7              category_name,
8              parent_category_id,
9              0 AS level,
10             CAST(category_name AS CHAR(500)) AS path
11         FROM Category
12         WHERE parent_category_id IS NULL
13
14         UNION ALL
15
16         -- Recursive: Get all child categories
17         SELECT
18             c.category_id,
19             c.category_name,
20             c.parent_category_id,
21             ct.level + 1,
22             CONCAT(ct.path, ' > ', c.category_name)
23         FROM Category c
24         INNER JOIN category_tree ct ON c.parent_category_id = ct.category_id
25         WHERE ct.level < 10 -- Prevent infinite loop, limit to 10 levels
26     )
27     SELECT
28         category_id,
29         category_name,
30         parent_category_id,
```

```
        level,
        path,
        REPEAT(' ', level) AS indent
    FROM category_tree
    ORDER BY path;
END$$
```

Related Tables: CATEGORY

Explanation: This procedure uses MySQL's recursive CTE (WITH RECURSIVE) to traverse the category hierarchy:

1. **Anchor Query**: Selects all root categories (parent_category_id IS NULL) at level 0
2. **Recursive Query**: Joins child categories to the tree, incrementing level
3. **Path Building**: Constructs a breadcrumb path (e.g., "Electronics > Phones > Smartphones")
4. **Recursion Limit**: Stops at level 10 to prevent infinite loops

Example Usage:

```
-- Get complete category hierarchy
CALL sp_get_category_hierarchy();

-- Result:
-- category_id | category_name | parent_id | level | path
-- 1           | Electronics   | NULL      | 0     | Electronics
-- 3           | Phones        | 1         | 1     | Electronics > Phones
-- 4           | Laptops       | 1         | 1     | Electronics > Laptops
-- 2           | Fashion       | NULL      | 0     | Fashion
-- 5           | Clothes       | 2         | 1     | Fashion > Clothes
```

9.4.2 Procedure: Get All Children of a Category

Purpose: Retrieves all descendant categories (children, grandchildren, etc.) of a specified parent category.

Listing 32: Procedure: sp_get_category_all_children

```
CREATE PROCEDURE sp_get_category_all_children(IN p_category_id INT)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Category WHERE category_id = p_category_id) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Category does not exist';
    END IF;

    WITH RECURSIVE children_tree AS (
```

```
        -- Anchor: Get the root category
        SELECT
            category_id,
            category_name,
            parent_category_id,
            0 AS level
        FROM Category
        WHERE category_id = p_category_id

        UNION ALL

        -- Recursive: Get all child categories at next level
        SELECT
            c.category_id,
            c.category_name,
            c.parent_category_id,
            ct.level + 1
        FROM Category c
        INNER JOIN children_tree ct ON c.parent_category_id = ct.category_id
        WHERE ct.level < 10
    )
    SELECT
        category_id,
        category_name,
        parent_category_id,
        level,
        REPEAT(' ', level) AS indent
    FROM children_tree
    ORDER BY level, category_name;
END$$
```

Related Tables: CATEGORY

Explanation: This procedure finds all descendants of a category:

1. Validates that the input category exists (raises error if not)
2. Anchor query retrieves the starting category at level 0
3. Recursive query finds all immediate children, then their children, etc.
4. Returns hierarchically structured results with indentation

Example Usage:

```
-- Get all children of Electronics (category_id = 1)
CALL sp_get_category_all_children(1);

-- Result:
-- category_id | category_name | parent_id | level
```

```
-- 1          | Electronics  | NULL     | 0
-- 3          | Phones       | 1        | 1
-- 4          | Laptops      | 1        | 1

-- Get all children of Fashion (category_id = 2)
CALL sp_get_category_all_children(2);

-- Result:
-- category_id | category_name | parent_id | level
-- 2          | Fashion      | NULL     | 0
-- 5          | Clothes      | 2        | 1
```

9.4.3 Procedure: Get All Parents of a Category

Purpose: Retrieves all ancestor categories (parent, grandparent, etc.) of a specified category, showing the reverse hierarchy path.

Listing 33: Procedure: sp_get_category_all_parents

```
CREATE PROCEDURE sp_get_category_all_parents(IN p_category_id INT)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Category WHERE category_id = p_category_id) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Category does not exist';
    END IF;

    WITH RECURSIVE parents_tree AS (
        -- Anchor: Get the root category
        SELECT
            category_id,
            category_name,
            parent_category_id,
            0 AS level
        FROM Category
        WHERE category_id = p_category_id

        UNION ALL

        -- Recursive: Get all parent categories at upper level
        SELECT
            c.category_id,
            c.category_name,
            c.parent_category_id,
            pt.level + 1
        FROM Category c
        INNER JOIN parents_tree pt ON pt.parent_category_id = c.category_id
        WHERE pt.level < 10
    )
    SELECT
```

```
        category_id,
        category_name,
        parent_category_id,
        level,
        REPEAT(' ', level) AS indent
    FROM parents_tree
    ORDER BY level DESC, category_name;
END$$
```

Related Tables: CATEGORY

Explanation: This procedure traverses the hierarchy upward to find all ancestors:

1. Validates category existence
2. Anchor retrieves the specified category
3. Recursive query follows parent_category_id chain upward
4. Returns results ordered from leaf to root (DESC level)

Example Usage:

```
-- Get all parents of Phones (category_id = 3)
CALL sp_get_category_all_parents(3);

-- Result (showing breadcrumb trail):
-- category_id | category_name | parent_id | level
-- 3          | Phones       | 1         | 0
-- 1          | Electronics  | NULL      | 1
-- This shows: Phones < Electronics

-- Get all parents of Clothes (category_id = 5)
CALL sp_get_category_all_parents(5);

-- Result:
-- category_id | category_name | parent_id | level
-- 5          | Clothes      | 2         | 0
-- 2          | Fashion      | NULL      | 1
-- This shows: Clothes < Fashion
```

9.5 Order Management Procedures

9.5.1 Procedure: Create Order from Cart

Purpose: Complex business logic procedure that creates an order from customer's shopping cart, validates inventory, applies vouchers, and updates related tables.

Listing 34: Procedure: sp_create_order_from_cart (Part 1)

```
CREATE PROCEDURE sp_create_order_from_cart(
    IN p_customer_id INT,
    IN p_shipping_id INT,
    IN p_voucher_id INT,
    IN p_shipping_address VARCHAR(255),
    IN p_payment_method VARCHAR(50),
    IN p_note TEXT,
    OUT p_order_id INT
)
BEGIN
    DECLARE v_subtotal DECIMAL(15, 2) DEFAULT 0;
    DECLARE v_shipping_fee DECIMAL(10, 2) DEFAULT 0;
    DECLARE v_discount DECIMAL(10, 2) DEFAULT 0;
    DECLARE v_total_amount DECIMAL(15, 2) DEFAULT 0;
    DECLARE v_cart_id INT;
    DECLARE v_cart_count INT DEFAULT 0;
    DECLARE v_discount_type VARCHAR(20);
    DECLARE v_discount_value DECIMAL(10, 2);
    DECLARE v_min_order_value DECIMAL(10, 2);
    DECLARE v_usage_limit INT;
    DECLARE v_used_count INT;

    -- Validation checks
    IF NOT EXISTS (SELECT 1 FROM Customer WHERE customer_id = p_customer_id) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Customer does not exist';
    END IF;

    IF NOT EXISTS (SELECT 1 FROM Shipping WHERE shipping_id = p_shipping_id
                AND status = 'Active') THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid or inactive shipping method';
    END IF;

    -- Get and validate cart
    SELECT cart_id INTO v_cart_id
    FROM Cart
    WHERE customer_id = p_customer_id;

    IF v_cart_id IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cart not found';
    END IF;

    SELECT COUNT(*) INTO v_cart_count
    FROM CartItem
    WHERE cart_id = v_cart_id;

    IF v_cart_count = 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cart is empty';
    END IF;

    -- Calculate order amounts
    SELECT COALESCE(SUM(ci.quantity * pi.price), 0)
    INTO v_subtotal
    FROM CartItem ci
    INNER JOIN ProductItem pi ON ci.item_id = pi.item_id
    WHERE ci.cart_id = v_cart_id;

    SELECT fee INTO v_shipping_fee
    FROM Shipping
    WHERE shipping_id = p_shipping_id;
```

Listing 35: Procedure: sp_create_order_from_cart (Part 2)

```

    -- Apply voucher if provided
    IF p_voucher_id IS NOT NULL THEN
        SELECT discount_type, discount_value, min_order_value,
               usage_limit, used_count
        INTO v_discount_type, v_discount_value, v_min_order_value,
             v_usage_limit, v_used_count
        FROM Voucher
        WHERE voucher_id = p_voucher_id
          AND status = 'Active'
          AND expired_date > CURDATE();

        IF v_discount_type IS NULL THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Voucher is invalid or expired';
        END IF;

        IF v_subtotal < v_min_order_value THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Order value does not meet voucher minimum';
        END IF;

        IF v_usage_limit > 0 AND v_used_count >= v_usage_limit THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Voucher usage limit reached';
        END IF;

        -- Calculate discount
        IF v_discount_type = 'Percentage' THEN
            SET v_discount = v_subtotal * (v_discount_value / 100);
        ELSEIF v_discount_type = 'Amount' THEN
            SET v_discount = v_discount_value;
        END IF;
```

```
34
35          -- Update voucher usage
36          UPDATE Voucher
37          SET used_count = used_count + 1
38          WHERE voucher_id = p_voucher_id;
39      END IF;
40
41      -- Calculate total
42      SET v_total_amount = v_subtotal + v_shipping_fee - v_discount;
43      IF v_total_amount < 0 THEN
44          SET v_total_amount = 0;
45      END IF;
46
47      -- Create order
48      INSERT INTO `Order` (
49          customer_id, shipping_id, voucher_id, status,
50          shipping_address, total_amount, payment_method, note
51      ) VALUES (
52          p_customer_id, p_shipping_id, p_voucher_id, 'Processing',
53          p_shipping_address, v_total_amount, p_payment_method, p_note
54      );
55
56      SET p_order_id = LAST_INSERT_ID();
57
58      -- Move cart items to order items
59      INSERT INTO OrderItem (order_id, item_id, shop_id, quantity, price_at_purchase)
60      SELECT p_order_id, ci.item_id, pi.shop_id, ci.quantity, pi.price
61      FROM CartItem ci
62      INNER JOIN ProductItem pi ON ci.item_id = pi.item_id
63      WHERE ci.cart_id = v_cart_id;
64
65      -- Clear cart
66      DELETE FROM CartItem WHERE cart_id = v_cart_id;
67
68      -- Update customer stats
69      UPDATE Customer
70      SET total_order = total_order + 1,
71          total_spent = total_spent + v_total_amount
72      WHERE customer_id = p_customer_id;
73  END$$
```

Related Tables: CUSTOMER, CART, CARTITEM, ORDER, ORDERITEM, SHIPPING, VOUCHER, PRODUCTITEM

Explanation: This is a complex transaction procedure that:

1. Validates customer, shipping method, and cart existence
2. Checks that cart contains items

3. Calculates subtotal from cart items
4. Validates and applies voucher (with min order value check)
5. Creates ORDER record with final total
6. Converts CARTITEM records to ORDERITEM records
7. Clears the shopping cart
8. Updates customer statistics (total_order, total_spent)

Example Usage:

```
-- Setup: Add items to Customer 1's cart
DELETE FROM CartItem WHERE cart_id = 1;
INSERT INTO CartItem (cart_id, item_id, quantity) VALUES (1, 1, 1);
INSERT INTO CartItem (cart_id, item_id, quantity) VALUES (1, 4, 2);

-- Verify cart contents before order creation
SELECT ci.cart_id, ci.item_id, ci.quantity, pi.price
FROM CartItem ci
INNER JOIN ProductItem pi ON ci.item_id = pi.item_id
WHERE ci.cart_id = 1;
-- Result: 2 rows (item 1, item 4)

-- Create order from customer 1's cart
CALL sp_create_order_from_cart(
    1,                          -- customer_id
    1,                          -- shipping_id
    NULL,                       -- voucher_id (no voucher)
    '123 Updated Street',       -- shipping_address
    'Banking',                  -- payment_method
    'Test from cart',           -- note
    @new_order_id              -- OUT parameter
);

SELECT @new_order_id AS order_id;
-- Result: New order_id (e.g., 5)

-- Verify order was created
SELECT order_id, customer_id, status, payment_method
FROM \`Order\` WHERE order_id = @new_order_id;
-- Result: status='Processing', payment_method='Banking'

-- Verify cart is empty after order creation
SELECT COUNT(*) AS remaining_items FROM CartItem WHERE cart_id = 1;
-- Result: 0 (cart cleared after order creation)
```

9.5.2 Procedure: Apply Voucher

Purpose: Validates a voucher code and calculates discount amount with comprehensive business rule checking.

Listing 36: Procedure: sp_apply_voucher

```
CREATE PROCEDURE sp_apply_voucher(
    IN p_voucher_code VARCHAR(50),
    IN p_order_amount DECIMAL(15, 2),
    OUT p_discount_amount DECIMAL(10, 2),
    OUT p_is_valid TINYINT,
    OUT p_message VARCHAR(255)
)
BEGIN
    DECLARE v_voucher_id INT;
    DECLARE v_discount_type VARCHAR(20);
    DECLARE v_discount_value DECIMAL(10, 2);
    DECLARE v_min_order_value DECIMAL(10, 2);
    DECLARE v_usage_limit INT;
    DECLARE v_used_count INT;
    DECLARE v_status VARCHAR(20);

    SET p_is_valid = 0;
    SET p_discount_amount = 0;

    -- Get voucher by code
    SELECT
        voucher_id, discount_type, discount_value, min_order_value,
        usage_limit, used_count, status
    INTO
        v_voucher_id, v_discount_type, v_discount_value, v_min_order_value,
        v_usage_limit, v_used_count, v_status
    FROM Voucher
    WHERE code = p_voucher_code;

    -- Validation chain
    IF v_voucher_id IS NULL THEN
        SET p_message = 'Voucher code not found';
    ELSEIF v_status != 'Active' THEN
        SET p_message = 'Voucher is not active';
    ELSEIF CURDATE() > (SELECT expired_date FROM Voucher WHERE voucher_id =
        v_voucher_id) THEN
        SET p_message = 'Voucher has expired';
    ELSEIF v_usage_limit > 0 AND v_used_count >= v_usage_limit THEN
        SET p_message = 'Voucher usage limit reached';
    ELSEIF p_order_amount < v_min_order_value THEN
        SET p_message = CONCAT('Minimum order value is ', v_min_order_value);
    ELSE
        -- Calculate discount
        CASE v_discount_type
```

```
            WHEN 'Percentage' THEN
                SET p_discount_amount = p_order_amount * (v_discount_value / 100);
            WHEN 'Amount' THEN
                SET p_discount_amount = v_discount_value;
        END CASE;

        -- Cap discount at order amount
        IF p_discount_amount > p_order_amount THEN
            SET p_discount_amount = p_order_amount;
        END IF;

        SET p_is_valid = 1;
        SET p_message = 'Voucher applied successfully';
    END IF;
END$$
```

Related Tables: VOUCHER

Explanation: This procedure validates vouchers through a series of checks:

1. Checks if voucher code exists in database
2. Verifies voucher is in 'Active' status
3. Checks expiration date against current date
4. Validates usage count hasn't exceeded limit
5. Validates order amount meets minimum requirement
6. Calculates discount (percentage or fixed amount)
7. Caps discount to not exceed order amount

Example Usage:

```
-- Test 1: Valid percentage voucher (DISCOUNT10)
SET @discount = 0;
SET @valid = FALSE;
SET @msg = '';
CALL sp_apply_voucher('DISCOUNT10', 500000, @discount, @valid, @msg);
SELECT @discount AS discount_amount, @valid AS is_valid, @msg AS message;
-- Result: discount=50000, is_valid=1, message='Voucher applied successfully'
-- DISCOUNT10 is 10% percentage discount

-- Test 2: Valid amount voucher (SAVE50) - Order meets minimum
CALL sp_apply_voucher('SAVE50', 300000, @discount, @valid, @msg);
SELECT @discount, @valid, @msg;
-- Result: discount=50000, is_valid=1, message='Voucher applied successfully'

```

```
-- Test 3: Order below minimum requirement
CALL sp_apply_voucher('SAVE50', 150000, @discount, @valid, @msg);
SELECT @msg;
-- Result: message mentions minimum order value not met

-- Test 4: Non-existent voucher
CALL sp_apply_voucher('NONEXISTENT', 500000, @discount, @valid, @msg);
SELECT @msg;
-- Result: 'Voucher code not found'
```

9.6 Shop Management Procedures

9.6.1 Procedure: Get Shop Revenue Report

Purpose: Comprehensive reporting procedure that generates shop revenue summary and top-selling products for a date range.

Listing 37: Procedure: sp_get_shop_revenue_report

```
CREATE PROCEDURE sp_get_shop_revenue_report(
    IN p_shop_id INT,
    IN p_from_date DATE,
    IN p_to_date DATE
)
BEGIN
    -- Validate shop exists
    IF NOT EXISTS (SELECT 1 FROM Shop WHERE shop_id = p_shop_id) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Shop does not exist';
    END IF;

    -- Main revenue summary
    SELECT
        s.shop_id,
        s.shop_code,
        s.shop_name,
        COUNT(DISTINCT o.order_id) AS total_orders,
        COUNT(DISTINCT oi.order_item_id) AS total_line_items,
        SUM(oi.quantity) AS total_units_sold,
        COALESCE(SUM(oi.quantity * oi.price_at_purchase), 0) AS total_revenue,
        COALESCE(AVG(oi.quantity * oi.price_at_purchase), 0) AS avg_order_value,
        s.rating AS shop_rating,
        p_from_date AS report_from,
        p_to_date AS report_to
    FROM Shop s
    LEFT JOIN OrderItem oi ON s.shop_id = oi.shop_id
    LEFT JOIN `Order` o ON oi.order_id = o.order_id
        AND o.order_date BETWEEN p_from_date AND DATE_ADD(p_to_date, INTERVAL 1 DAY)
        AND o.status != 'Cancelled'
```

```
    WHERE s.shop_id = p_shop_id
    GROUP BY s.shop_id, s.shop_code, s.shop_name, s.rating;

    -- Top 10 selling product items
    SELECT
        p.product_id,
        p.product_name,
        pi.color,
        pi.type,
        COUNT(oi.order_item_id) AS times_ordered,
        SUM(oi.quantity) AS total_quantity_sold,
        SUM(oi.quantity * oi.price_at_purchase) AS item_revenue,
        ROUND(SUM(oi.quantity * oi.price_at_purchase) / COUNT(oi.order_item_id), 2)
            AS avg_item_value
    FROM Product p
    INNER JOIN ProductItem pi ON p.product_id = pi.product_id
    INNER JOIN OrderItem oi ON pi.item_id = oi.item_id
    INNER JOIN `Order` o ON oi.order_id = o.order_id
        AND o.order_date BETWEEN p_from_date AND DATE_ADD(p_to_date, INTERVAL 1 DAY)
        AND o.status != 'Cancelled'
    WHERE pi.shop_id = p_shop_id
    GROUP BY p.product_id, p.product_name, pi.color, pi.type
    ORDER BY item_revenue DESC
    LIMIT 10;
END$$
```

Related Tables: SHOP, ORDER, ORDERITEM, PRODUCT, PRODUCTITEM

Explanation: This reporting procedure generates two result sets:

Result Set 1 - Shop Summary:

- Total orders during period
- Line items and units sold
- Revenue metrics (total and average)
- Current shop rating
- Date range of report

Result Set 2 - Top 10 Products:

- Product and variant details (color, type)
- Order frequency
- Revenue contribution

- Average item value

Example Usage:

```
-- Get revenue report for shop 4 for full year 2025
CALL sp_get_shop_revenue_report(4, '2025-01-01', '2025-12-31');

-- Result Set 1 (Summary):
-- shop_id | shop_code | shop_name | total_orders | total_units | revenue
-- 4       | SHOP00004 | Shop ABC | 1            | 2           | 2,200,000

-- Result Set 2 (Top Products):
-- product_name | color | type | times_ordered | qty_sold | revenue
-- iPhone 15    | Red   | 128GB | 1            | 1        | 1,000,000
-- iPhone 15    | Blue  | 256GB | 1            | 1        | 1,200,000

-- Get revenue report for shop 5
CALL sp_get_shop_revenue_report(5, '2025-01-01', '2025-12-31');

-- Result Set 1:
-- shop_id | shop_code | shop_name | total_orders | total_units | revenue
-- 5       | SHOP00005 | Shop XYZ | 1            | 1           | 25,000

-- Result Set 2:
-- product_name | color | type | times_ordered | qty_sold | revenue
-- T-Shirt      | White | Size M | 1           | 1        | 25,000
```

9.7 Product CRUD Procedures

9.7.1 Procedure: Add Product

Purpose: Creates a new product in the system with validation for shop and category existence.

Listing 38: Procedure: sp_add_product

```
CREATE PROCEDURE sp_add_product(
    IN p_shop_id INT,
    IN p_category_id INT,
    IN p_product_name VARCHAR(255),
    IN p_description TEXT,
    IN p_image VARCHAR(255),
    OUT p_product_id INT,
    OUT p_status VARCHAR(50)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET p_product_id = -1;
        SET p_status = 'Error: Invalid shop or category ID';
    END;
```

```
16
17      SET p_product_id = -1;
18      SET p_status = 'Error';
19
20      -- Validate shop exists
21      IF NOT EXISTS (SELECT 1 FROM Shop WHERE shop_id = p_shop_id) THEN
22          SET p_status = 'Error: Shop not found';
23      ELSEIF NOT EXISTS (SELECT 1 FROM Category
24                         WHERE category_id = p_category_id) THEN
25          SET p_status = 'Error: Category not found';
26      ELSE
27          -- Insert product
28          INSERT INTO Product (shop_id, category_id, product_name,
29                               description, image, status)
30          VALUES (p_shop_id, p_category_id, p_product_name,
31                  p_description, p_image, 'In stock');
32
33          SET p_product_id = LAST_INSERT_ID();
34          SET p_status = 'Success';
35      END IF;
36  END$$
```

Related Tables: PRODUCT, SHOP, CATEGORY

Explanation: This procedure creates a new product with error handling:

1. **EXIT HANDLER**: Catches SQL exceptions and returns error status
2. **Shop Validation**: Verifies shop_id exists in Shop table
3. **Category Validation**: Verifies category_id exists in Category table
4. **Product Creation**: Inserts new product with default status 'In stock'
5. **OUT Parameters**: Returns new product_id and status message

Example Usage:

```
1  -- Add new product to Shop 4, Category 3 (Phones)
2  CALL sp_add_product(
3      4,                          -- shop_id
4      3,                          -- category_id
5      'Samsung Galaxy S24',       -- product_name
6      'Latest Samsung flagship',  -- description
7      'samsung-s24.jpg',          -- image
8      @product_id,                -- OUT: product_id
9      @status                     -- OUT: status message
10 );
11
12 SELECT @product_id, @status;
```

```
-- Result: @product_id = 5, @status = 'Success'

-- Verify product was created
SELECT product_id, product_name, status
FROM Product WHERE product_id = @product_id;
-- Result: Samsung Galaxy S24, In stock

-- Test invalid shop ID
CALL sp_add_product(
    999,                        -- invalid shop_id
    3,
    'Test Product',
    'Description',
    'test.jpg',
    @product_id,
    @status
);

SELECT @status;
-- Result: 'Error: Shop not found'
```

9.7.2 Procedure: Update Product

Purpose: Updates existing product information with NULL-safe field updates.

Listing 39: Procedure: sp_update_product

```
CREATE PROCEDURE sp_update_product(
    IN p_product_id INT,
    IN p_product_name VARCHAR(255),
    IN p_description TEXT,
    IN p_image VARCHAR(255),
    IN p_status VARCHAR(50),
    OUT p_success BOOLEAN,
    OUT p_message VARCHAR(100)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET p_success = FALSE;
        SET p_message = 'Error: Database error';
    END;

    IF NOT EXISTS (SELECT 1 FROM Product WHERE product_id = p_product_id) THEN
        SET p_success = FALSE;
        SET p_message = 'Error: Product not found';
    ELSE
        UPDATE Product
        SET product_name = COALESCE(p_product_name, product_name),
            description = COALESCE(p_description, description),
```

```
            image = COALESCE(p_image, image),
            status = COALESCE(p_status, status)
        WHERE product_id = p_product_id;

        SET p_success = TRUE;
        SET p_message = 'Product updated successfully';
    END IF;
END$$
```

Related Tables: PRODUCT

Explanation: This procedure demonstrates selective field updates:

1. **COALESCE Pattern**: Updates only non-NULL input fields
2. **Existence Check**: Validates product_id before update
3. **Error Handling**: Returns success flag and descriptive message
4. **Partial Updates**: Allows updating individual fields without affecting others

Example Usage:

```
-- Update only product name and status
CALL sp_update_product(
    1,                          -- product_id
    'iPhone 15 Pro Max',        -- new name
    NULL,                       -- keep existing description
    NULL,                       -- keep existing image
    'Out of stock',             -- new status
    @success,
    @message
);

SELECT @success, @message;
-- Result: TRUE, 'Product updated successfully'

-- Verify update
SELECT product_name, status FROM Product WHERE product_id = 1;
-- Result: iPhone 15 Pro Max, Out of stock

-- Update only description
CALL sp_update_product(
    1,
    NULL,                       -- keep existing name
    'Updated description text', -- new description
    NULL,
    NULL,
    @success,
    @message
```

```
);
```

9.7.3 Procedure: Delete Product

Purpose: Deletes a product and all related product variants with CASCADE behavior.

Listing 40: Procedure: sp_delete_product

```
CREATE PROCEDURE sp_delete_product(
    IN p_product_id INT,
    OUT p_success BOOLEAN,
    OUT p_message VARCHAR(100)
)
BEGIN
    DECLARE v_affected_items INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET p_success = FALSE;
        SET p_message = 'Error: Database error';
    END;

    IF NOT EXISTS (SELECT 1 FROM Product WHERE product_id = p_product_id) THEN
        SET p_success = FALSE;
        SET p_message = 'Error: Product not found';
    ELSE
        -- Get count of affected items before deletion
        SELECT COUNT(*) INTO v_affected_items
        FROM ProductItem
        WHERE product_id = p_product_id;

        -- Delete product (CASCADE deletes ProductItems)
        DELETE FROM Product WHERE product_id = p_product_id;

        SET p_success = TRUE;
        SET p_message = CONCAT('Product deleted. Removed ',
                               v_affected_items, ' variant(s)');
    END IF;
END$$
```

Related Tables: PRODUCT, PRODUCTITEM

Explanation: This procedure handles cascading deletion:

1. **Pre-deletion Count**: Records number of variants before deletion
2. **CASCADE Behavior**: Foreign key constraints automatically delete related ProductItems

3. **Informative Message**: Returns count of deleted variants
4. **Transactional Safety**: EXIT HANDLER ensures atomicity

Example Usage:

```
-- Check product variants before deletion
SELECT COUNT(*) AS variant_count
FROM ProductItem
WHERE product_id = 1;
-- Result: 2 variants (Red 128GB, Blue 256GB)

-- Delete product
CALL sp_delete_product(1, @success, @message);

SELECT @success, @message;
-- Result: TRUE, 'Product deleted. Removed 2 variant(s)'

-- Verify deletion
SELECT COUNT(*) FROM Product WHERE product_id = 1;
-- Result: 0

SELECT COUNT(*) FROM ProductItem WHERE product_id = 1;
-- Result: 0 (cascaded deletion)
```

9.7.4 Procedure: Add Product Item (Variant)

Purpose: Adds a new product variant with color, type, price, and stock information.

Listing 41: Procedure: sp_add_product_item

```
CREATE PROCEDURE sp_add_product_item(
    IN p_product_id INT,
    IN p_shop_id INT,
    IN p_color VARCHAR(50),
    IN p_type VARCHAR(50),
    IN p_price DECIMAL(15,2),
    IN p_stock INT,
    IN p_image_url VARCHAR(255),
    OUT p_item_id INT,
    OUT p_status VARCHAR(50)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET p_item_id = -1;
        SET p_status = 'Error: Invalid input';
    END;

    SET p_item_id = -1;
    SET p_status = 'Error';
```

```
21
22      IF NOT EXISTS (SELECT 1 FROM Product WHERE product_id = p_product_id) THEN
23          SET p_status = 'Error: Product not found';
24      ELSEIF p_price <= 0 THEN
25          SET p_status = 'Error: Price must be > 0';
26      ELSEIF p_stock < 0 THEN
27          SET p_status = 'Error: Stock cannot be negative';
28      ELSE
29          INSERT INTO ProductItem (product_id, shop_id, color, type,
30                                   price, stock, image_url)
31          VALUES (p_product_id, p_shop_id, p_color, p_type,
32                  p_price, p_stock, p_image_url);
33
34          SET p_item_id = LAST_INSERT_ID();
35          SET p_status = 'Success';
36      END IF;
37  END$$
```

Related Tables: PRODUCTITEM, PRODUCT, SHOP

Explanation: This procedure creates product variants with validation:

1. **Product Existence**: Validates parent product exists
2. **Price Validation**: Ensures price > 0
3. **Stock Validation**: Ensures stock $>= 0$
4. **Variant Details**: Stores color, type (size/capacity), image

Example Usage:

```
1  -- Add new variant: iPhone 15 Green 512GB
2  CALL sp_add_product_item(
3      1,                          -- product_id (iPhone 15)
4      4,                          -- shop_id
5      'Green',                    -- color
6      '512GB',                    -- type
7      1400000.00,                 -- price
8      15,                         -- stock
9      'iphone15-green-512.jpg', -- image_url
10     @item_id,
11     @status
12 );
13
14 SELECT @item_id, @status;
15 -- Result: @item_id = 5, @status = 'Success'
16
17 -- Verify variant was created
18 SELECT item_id, color, type, price, stock
```

```
FROM ProductItem WHERE item_id = @item_id;
-- Result: 5, Green, 512GB, 1400000.00, 15

-- Test invalid price
CALL sp_add_product_item(
    1, 4, 'Black', '128GB',
    -100.00,                    -- invalid price
    10, 'test.jpg',
    @item_id, @status
);

SELECT @status;
-- Result: 'Error: Price must be > 0'
```

9.7.5 Procedure: Update Product Item

Purpose: Updates product variant price, stock, and image with validation.

Listing 42: Procedure: sp_update_product_item

```
CREATE PROCEDURE sp_update_product_item(
    IN p_item_id INT,
    IN p_price DECIMAL(15,2),
    IN p_stock INT,
    IN p_image_url VARCHAR(255),
    OUT p_success BOOLEAN,
    OUT p_message VARCHAR(100)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET p_success = FALSE;
        SET p_message = 'Error: Database error';
    END;

    IF NOT EXISTS (SELECT 1 FROM ProductItem WHERE item_id = p_item_id) THEN
        SET p_success = FALSE;
        SET p_message = 'Error: Item not found';
    ELSEIF p_price IS NOT NULL AND p_price <= 0 THEN
        SET p_success = FALSE;
        SET p_message = 'Error: Price must be > 0';
    ELSEIF p_stock IS NOT NULL AND p_stock < 0 THEN
        SET p_success = FALSE;
        SET p_message = 'Error: Stock cannot be negative';
    ELSE
        UPDATE ProductItem
        SET price = COALESCE(p_price, price),
            stock = COALESCE(p_stock, stock),
            image_url = COALESCE(p_image_url, image_url)
        WHERE item_id = p_item_id;
```

```
31
32          SET p_success = TRUE;
33          SET p_message = 'Item updated successfully';
34      END IF;
35  END$$
```

Related Tables: PRODUCTITEM

Explanation: This procedure demonstrates selective updates with validation:

1. **COALESCE Updates**: Updates only provided fields
2. **Pre-update Validation**: Checks constraints before UPDATE
3. **Business Rules**: Price must be positive, stock non-negative

Example Usage:

```
1  -- Update only price and stock
2  CALL sp_update_product_item(
3      1,                          -- item_id
4      1100000.00,                 -- new price
5      50,                         -- new stock
6      NULL,                       -- keep existing image
7      @success,
8      @message
9  );
10
11 SELECT @success, @message;
12 -- Result: TRUE, 'Item updated successfully'
13
14 -- Update only stock
15 CALL sp_update_product_item(
16     1,
17     NULL,                       -- keep existing price
18     45,                         -- decrease stock
19     NULL,
20     @success,
21     @message
22 );
```

9.7.6 Procedure: Delete Product Item

Purpose: Deletes a specific product variant.

Listing 43: Procedure: sp_delete_product_item

```
1  CREATE PROCEDURE sp_delete_product_item(
2      IN p_item_id INT,
```

```
    OUT p_success BOOLEAN,
    OUT p_message VARCHAR(100)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SET p_success = FALSE;
        SET p_message = 'Error: Database error';
    END;

    IF NOT EXISTS (SELECT 1 FROM ProductItem WHERE item_id = p_item_id) THEN
        SET p_success = FALSE;
        SET p_message = 'Error: Item not found';
    ELSE
        DELETE FROM ProductItem WHERE item_id = p_item_id;

        SET p_success = TRUE;
        SET p_message = 'Item deleted successfully';
    END IF;
END$$
```

Related Tables: PRODUCTITEM

Explanation: Simple deletion procedure with existence validation and error handling.

Example Usage:

```
-- Delete variant
CALL sp_delete_product_item(2, @success, @message);

SELECT @success, @message;
-- Result: TRUE, 'Item deleted successfully'

-- Verify deletion
SELECT COUNT(*) FROM ProductItem WHERE item_id = 2;
-- Result: 0
```

9.8 Product Search Procedure

9.8.1 Procedure: Get Product List with Filters

Purpose: Dynamic product search with filters, sorting, aggregations, and pagination using dynamic SQL.

Listing 44: Procedure: sp_get_product_list (Excerpt)

```
CREATE PROCEDURE sp_get_product_list(
    IN p_search VARCHAR(255),
```

```
    IN p_category_id INT,
    IN p_shop_id INT,
    IN p_min_price DECIMAL(15,2),
    IN p_max_price DECIMAL(15,2),
    IN p_status VARCHAR(50),
    IN p_sort_by VARCHAR(50),
    IN p_sort_order VARCHAR(10),
    IN p_page INT,
    IN p_limit INT,
    OUT p_total_count INT
)
BEGIN
    DECLARE v_offset INT;
    DECLARE v_order_clause VARCHAR(100);

    SET v_offset = (p_page - 1) * p_limit;

    -- Build ORDER clause
    SET v_order_clause = CASE p_sort_by
        WHEN 'price' THEN CONCAT('min_price ', COALESCE(p_sort_order, 'ASC'))
        WHEN 'rating' THEN CONCAT('avg_rating ', COALESCE(p_sort_order, 'DESC'))
        WHEN 'created_at' THEN CONCAT('p.created_at ', COALESCE(p_sort_order,
            'DESC'))
        ELSE 'p.created_at DESC'
    END;

    -- Count total matching records
    SELECT COUNT(DISTINCT p.product_id) INTO p_total_count
    FROM Product p
    INNER JOIN Shop s ON p.shop_id = s.shop_id
    INNER JOIN Category c ON p.category_id = c.category_id
    LEFT JOIN ProductItem pi ON p.product_id = pi.product_id
    WHERE 1=1
        AND (p_search IS NULL OR p.product_name LIKE CONCAT('%', p_search, '%')
             OR p.description LIKE CONCAT('%', p_search, '%'))
        AND (p_category_id IS NULL OR p.category_id = p_category_id)
        AND (p_shop_id IS NULL OR p.shop_id = p_shop_id)
        AND (p_status IS NULL OR p.status = p_status);

    -- Dynamic query construction
    SET @sql = CONCAT('
        SELECT
            p.product_id,
            p.product_name,
            p.description,
            p.image,
            p.status,
            p.created_at,
            p.shop_id,
            s.shop_name,
```

```
            s.rating as shop_rating,
            c.category_id,
            c.category_name,
            MIN(pi.price) as min_price,
            MAX(pi.price) as max_price,
            SUM(pi.stock) as total_stock,
            COUNT(DISTINCT pi.item_id) as variant_count,
            COALESCE((
                SELECT AVG(r.rating)
                FROM Review r
                WHERE r.target_id = p.product_id AND r.target_type = "Product"
            ), 0) as avg_rating,
            COALESCE((
                SELECT COUNT(*)
                FROM Review r
                WHERE r.target_id = p.product_id AND r.target_type = "Product"
            ), 0) as review_count
        FROM Product p
        INNER JOIN Shop s ON p.shop_id = s.shop_id
        INNER JOIN Category c ON p.category_id = c.category_id
        LEFT JOIN ProductItem pi ON p.product_id = pi.product_id
        WHERE 1=1',
        CASE WHEN p_search IS NOT NULL THEN
            CONCAT(' AND (p.product_name LIKE "%', p_search, '%" OR p.description
                LIKE "%', p_search, '%")')
        ELSE '' END,
        CASE WHEN p_category_id IS NOT NULL THEN CONCAT(' AND p.category_id = ',
            p_category_id)
        ELSE '' END,
        CASE WHEN p_shop_id IS NOT NULL THEN CONCAT(' AND p.shop_id = ', p_shop_id)
        ELSE '' END,
        CASE WHEN p_status IS NOT NULL THEN CONCAT(' AND p.status = "', p_status,
            '"')
        ELSE '' END,
        ' GROUP BY p.product_id, p.product_name, p.description, p.image, p.status,
                  p.created_at, p.shop_id, s.shop_name, s.rating, c.category_id,
                      c.category_name',
        CASE WHEN p_min_price IS NOT NULL THEN CONCAT(' HAVING min_price >= ',
            p_min_price)
        ELSE '' END,
        CASE WHEN p_max_price IS NOT NULL THEN
            CONCAT(CASE WHEN p_min_price IS NOT NULL THEN ' AND' ELSE ' HAVING' END,
                  ' max_price <= ', p_max_price)
        ELSE '' END,
        ' ORDER BY ', v_order_clause,
        ' LIMIT ', p_limit, ' OFFSET ', v_offset
    );

    PREPARE stmt FROM @sql;
    EXECUTE stmt;
```

```
    DEALLOCATE PREPARE stmt;
END$$
```

Related Tables: PRODUCT, PRODUCTITEM, SHOP, CATEGORY, REVIEW

Explanation: This advanced procedure demonstrates:

1. **Dynamic SQL Construction**: Builds query based on provided filters
2. **Multiple Filters**: Search text, category, shop, status, price range
3. **Aggregations**: MIN/MAX price, SUM stock, COUNT variants, AVG rating
4. **Subqueries**: Rating and review count calculated via subselects
5. **Sorting**: Dynamic sort order based on price/rating/date
6. **Pagination**: Offset and limit for result set paging
7. **Total Count**: OUT parameter returns total matching records

Example Usage:

```
-- Search smartphones under 10M, sorted by rating, page 1
DECLARE @total INT;
CALL sp_get_product_list(
    'iPhone',      -- search term
    3,             -- category_id (Smartphones)
    NULL,          -- shop_id (any)
    0,             -- min_price
    10000000,      -- max_price
    'In stock',    -- status
    'rating',      -- sort by rating
    'DESC',        -- descending
    1,             -- page 1
    20,            -- 20 items per page
    @total         -- OUT: total count
);

SELECT @total; -- e.g., 45 total matches
```

10 Database Triggers

This section documents the automated database triggers that enforce business rules, maintain data integrity, and handle derived attributes.

10.1 Order Management Triggers

10.1.1 Trigger: Order Item Before Insert

Purpose: Validates stock availability and sets derived values (price_at_purchase, shop_id) before inserting order items.

Listing 45: Trigger: trg_order_item_before_insert

```
CREATE TRIGGER trg_order_item_before_insert
BEFORE INSERT ON OrderItem
FOR EACH ROW
BEGIN
    DECLARE v_item_price DECIMAL(10, 2);
    DECLARE v_item_stock INT;
    DECLARE v_shop_id INT;

    -- Get product item details
    SELECT price, stock, shop_id
    INTO v_item_price, v_item_stock, v_shop_id
    FROM ProductItem
    WHERE item_id = NEW.item_id;

    -- Validate item exists
    IF v_item_price IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Product item does not exist';
    END IF;

    -- Check stock availability
    IF v_item_stock < NEW.quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient stock for this product item';
    END IF;

    -- Set price at purchase if not provided
    IF NEW.price_at_purchase IS NULL OR NEW.price_at_purchase = 0 THEN
        SET NEW.price_at_purchase = v_item_price;
    END IF;

    -- Set shop_id if not provided
    IF NEW.shop_id IS NULL OR NEW.shop_id = 0 THEN
        SET NEW.shop_id = v_shop_id;
    END IF;
END$$
```

Trigger Type: BEFORE INSERT

Related Tables: ORDERITEM, PRODUCTITEM

Business Logic: This trigger enforces critical order integrity:

1. **Existence Validation**: Ensures item_id references valid ProductItem
2. **Stock Validation**: Prevents ordering more items than available stock
3. **Price Capture**: Records current price as price_at_purchase for historical accuracy
4. **Shop Association**: Automatically links order item to correct shop

Example Scenario:

```
-- Scenario 1: Valid order item insertion
-- ProductItem 1: price=1,000,000, stock=10
INSERT INTO OrderItem (order_id, item_id, quantity)
VALUES (1, 1, 2);
-- Success: price_at_purchase=1,000,000, shop_id set automatically

-- Scenario 2: Insufficient stock
-- ProductItem 2: stock=5
INSERT INTO OrderItem (order_id, item_id, quantity)
VALUES (1, 2, 10);
-- Error: 'Insufficient stock for this product item'

-- Scenario 3: Invalid item_id
INSERT INTO OrderItem (order_id, item_id, quantity)
VALUES (1, 999, 1);
-- Error: 'Product item does not exist'
```

10.1.2 Trigger: Order Item After Insert

Purpose: Decreases product stock after order item insertion and updates product status if out of stock.

Listing 46: Trigger: trg_order_item_after_insert

```
CREATE TRIGGER trg_order_item_after_insert
AFTER INSERT ON OrderItem
FOR EACH ROW
BEGIN
    DECLARE v_product_id INT;
    DECLARE v_total_stock INT;

    -- Decrease product item stock
    UPDATE ProductItem
    SET stock = stock - NEW.quantity
    WHERE item_id = NEW.item_id;

    -- Get product_id and check total stock
    SELECT product_id INTO v_product_id
    FROM ProductItem
```

```
    WHERE item_id = NEW.item_id;

    SELECT SUM(stock) INTO v_total_stock
    FROM ProductItem
    WHERE product_id = v_product_id;

    -- Update product status if all variants are out of stock
    IF v_total_stock <= 0 THEN
        UPDATE Product
        SET status = 'Out of stock'
        WHERE product_id = v_product_id;
    END IF;
END$$
```

Trigger Type: AFTER INSERT

Related Tables: ORDERITEM, PRODUCTITEM, PRODUCT

Business Logic: Maintains inventory consistency:

1. **Stock Deduction**: Automatically decreases ProductItem stock by ordered quantity
2. **Aggregate Check**: Calculates total stock across all product variants
3. **Status Update**: Marks product as 'Out of stock' when all variants depleted

Example Scenario:

```
-- Before: iPhone 15 has 2 variants
-- Variant 1 (Red): stock=10
-- Variant 2 (Blue): stock=5

-- Customer orders 10 Red iPhones
INSERT INTO OrderItem (order_id, item_id, quantity)
VALUES (1, 1, 10);

-- After trigger execution:
-- Variant 1 (Red): stock=0
-- Variant 2 (Blue): stock=5
-- Product status: Still 'Active' (total stock = 5)

-- Customer orders 5 Blue iPhones
INSERT INTO OrderItem (order_id, item_id, quantity)
VALUES (1, 2, 5);

-- After trigger execution:
-- Variant 1 (Red): stock=0
-- Variant 2 (Blue): stock=0
-- Product status: 'Out of stock' (total stock = 0)
```

10.1.3 Trigger: Order After Update (Stock Restoration)

Purpose: Restores product stock when an order is cancelled, using cursor to iterate through order items.

Listing 47: Trigger: trg_order_after_update

```
CREATE TRIGGER trg_order_after_update
AFTER UPDATE ON `Order`
FOR EACH ROW
BEGIN
    DECLARE v_product_id INT;
    DECLARE v_total_stock INT;

    -- If order status changed from non-Cancelled to Cancelled
    IF OLD.status != 'Cancelled' AND NEW.status = 'Cancelled' THEN
        BEGIN
            DECLARE done INT DEFAULT FALSE;
            DECLARE v_item_id INT;
            DECLARE v_quantity INT;

            DECLARE item_cursor CURSOR FOR
                SELECT item_id, quantity
                FROM OrderItem
                WHERE order_id = NEW.order_id;

            DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

            OPEN item_cursor;

            restore_loop: LOOP
                FETCH item_cursor INTO v_item_id, v_quantity;
                IF done THEN
                    LEAVE restore_loop;
                END IF;

                -- Restore stock for this item
                UPDATE ProductItem
                SET stock = stock + v_quantity
                WHERE item_id = v_item_id;

                -- Get product_id
                SELECT product_id INTO v_product_id
                FROM ProductItem
                WHERE item_id = v_item_id;

                -- Check total stock for this product
                SELECT SUM(stock) INTO v_total_stock
                FROM ProductItem
                WHERE product_id = v_product_id;

```

```
                -- Update product status back to Active if stock available
                IF v_total_stock > 0 THEN
                    UPDATE Product
                    SET status = 'Active'
                    WHERE product_id = v_product_id
                      AND status = 'Out of stock';
                END IF;
            END LOOP;

            CLOSE item_cursor;
        END;
    END IF;
END$$
```

Trigger Type: AFTER UPDATE

Related Tables: ORDER, ORDERITEM, PRODUCTITEM, PRODUCT

Business Logic: Handles order cancellation with cursor-based iteration:

1. **Status Change Detection**: Only executes when order status changes to 'Cancelled'
2. **Cursor Loop**: Iterates through all OrderItems in the cancelled order
3. **Stock Restoration**: Adds back ordered quantity to ProductItem stock
4. **Status Reactivation**: Changes Product status from 'Out of stock' to 'Active'

Example Scenario:

```
-- Order 1 contains:
-- Item 1 (Red iPhone): quantity=10
-- Item 2 (Blue iPhone): quantity=5

-- Before cancellation:
-- Variant 1: stock=0
-- Variant 2: stock=0
-- Product status: 'Out of stock'

-- Cancel the order
UPDATE `Order` SET status = 'Cancelled' WHERE order_id = 1;

-- After trigger execution:
-- Variant 1: stock=10 (restored)
-- Variant 2: stock=5 (restored)
-- Product status: 'Active' (reactivated)
```

10.2 Review Management Triggers

10.2.1 Trigger: Review Before Insert (Verified Purchase)

Purpose: Enforces verified purchase requirement - customers can only review products/shops from delivered orders.

Listing 48: Trigger: trg_review_before_insert

```
CREATE TRIGGER trg_review_before_insert
BEFORE INSERT ON Review
FOR EACH ROW
BEGIN
    -- Enforce rating between 1 and 5
    IF NEW.rating < 1 OR NEW.rating > 5 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Rating must be between 1 and 5';
    END IF;

    -- Validate based on target type
    IF NEW.target_type = 'Product' THEN
        -- Check if customer purchased this product
        IF NOT EXISTS (
            SELECT 1
            FROM OrderItem oi
            INNER JOIN `Order` o ON oi.order_id = o.order_id
            INNER JOIN ProductItem pi ON oi.item_id = pi.item_id
            WHERE pi.product_id = NEW.target_id
              AND o.customer_id = NEW.customer_id
              AND o.status = 'Delivered'
        ) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Can only review products from delivered orders';
        END IF;
    ELSEIF NEW.target_type = 'Shop' THEN
        -- Check if customer purchased from this shop
        IF NOT EXISTS (
            SELECT 1
            FROM OrderItem oi
            INNER JOIN `Order` o ON oi.order_id = o.order_id
            WHERE oi.shop_id = NEW.target_id
              AND o.customer_id = NEW.customer_id
              AND o.status = 'Delivered'
        ) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Can only review shops you have purchased from';
        END IF;
    END IF;
END$$
```

Trigger Type: BEFORE INSERT

Related Tables: REVIEW, ORDER, ORDERITEM, PRODUCTITEM

Business Logic: Implements verified purchase policy:

1. **Rating Validation**: Ensures rating is between 1 and 5
2. **Product Reviews**: Requires customer to have received (status='Delivered') the product
3. **Shop Reviews**: Requires customer to have completed purchase from the shop
4. **Fraud Prevention**: Prevents fake reviews from non-customers

Example Scenario:

```
-- Scenario 1: Valid product review (customer received product)
-- Customer 1 has Order 1 (status='Delivered') containing Product 1
INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
VALUES (1, 'Product', 1, 5, 'Great product!');
-- Success

-- Scenario 2: Invalid review (customer never purchased product)
INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
VALUES (2, 'Product', 1, 5, 'Fake review');
-- Error: 'Can only review products from delivered orders'

-- Scenario 3: Invalid rating
INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
VALUES (1, 'Product', 1, 10, 'Rating too high');
-- Error: 'Rating must be between 1 and 5'

-- Scenario 4: Valid shop review
INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
VALUES (1, 'Shop', 4, 4, 'Good service');
-- Success (Customer 1 bought from Shop 4)
```

10.2.2 Trigger: Review After Insert (Rating Update)

Purpose: Automatically updates product or shop rating after a new review is submitted.

Listing 49: Trigger: trg_review_after_insert

```
CREATE TRIGGER trg_review_after_insert
AFTER INSERT ON Review
FOR EACH ROW
BEGIN
    DECLARE v_avg_rating DECIMAL(3, 2);

    IF NEW.target_type = 'Shop' THEN
```

```
        -- Update shop rating
        SELECT AVG(rating) INTO v_avg_rating
        FROM Review
        WHERE target_type = 'Shop' AND target_id = NEW.target_id;

        UPDATE Shop
        SET rating = COALESCE(v_avg_rating, 0)
        WHERE shop_id = NEW.target_id;

    ELSEIF NEW.target_type = 'Product' THEN
        -- Update product rating
        SELECT AVG(rating) INTO v_avg_rating
        FROM Review
        WHERE target_type = 'Product' AND target_id = NEW.target_id;

        UPDATE Product
        SET rating = COALESCE(v_avg_rating, 0)
        WHERE product_id = NEW.target_id;
    END IF;
END$$
```

Trigger Type: AFTER INSERT

Related Tables: REVIEW, SHOP, PRODUCT

Business Logic: Maintains denormalized rating data for performance:

1. **Average Calculation**: Computes AVG(rating) from all reviews
2. **Shop Rating Update**: Updates Shop.rating for shop reviews
3. **Product Rating Update**: Updates Product.rating for product reviews
4. **Real-time Sync**: Rating immediately reflects new review

Example Scenario:

```
-- Before: Product 1 has 2 reviews (5 stars, 4 stars)
-- Product.rating = 4.50

-- Customer adds 3-star review
INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
VALUES (3, 'Product', 1, 3, 'Average product');

-- After trigger execution:
-- Product 1 now has 3 reviews: (5 + 4 + 3) / 3 = 4.00
-- Product.rating = 4.00

SELECT rating FROM Product WHERE product_id = 1;
```

```
-- Result: 4.00
```

10.3 Inventory Management Triggers

10.3.1 Trigger: Product Item Before Update

Purpose: Validates business constraints before updating product item (prevents negative stock/price).

Listing 50: Trigger: trg_product_item_before_update

```
CREATE TRIGGER trg_product_item_before_update
BEFORE UPDATE ON ProductItem
FOR EACH ROW
BEGIN
    -- Prevent negative stock
    IF NEW.stock < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Product item stock cannot be negative';
    END IF;

    -- Prevent negative price
    IF NEW.price < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Product item price cannot be negative';
    END IF;
END$$
```

Trigger Type: BEFORE UPDATE

Related Tables: PRODUCTITEM

Business Logic: Enforces CHECK constraints at trigger level:

1. **Stock Constraint**: Prevents stock from becoming negative
2. **Price Constraint**: Prevents price from becoming negative
3. **Data Integrity**: Ensures business rules are enforced at database level

Example Scenario:

```
-- Scenario 1: Valid stock update
UPDATE ProductItem SET stock = 50 WHERE item_id = 1;
-- Success

-- Scenario 2: Attempt negative stock
UPDATE ProductItem SET stock = -10 WHERE item_id = 1;
```

```
-- Error: 'Product item stock cannot be negative'

-- Scenario 3: Attempt negative price
UPDATE ProductItem SET price = -1000 WHERE item_id = 1;
-- Error: 'Product item price cannot be negative'
```

10.4 Account Management Triggers

10.4.1 Trigger: Account After Insert (Subclass Creation)

Purpose: Automatically creates corresponding Customer/Shop/Admin record and related entities (Cart for Customer) when Account is inserted.

Listing 51: Trigger: trg_account_after_insert

```
CREATE TRIGGER trg_account_after_insert
AFTER INSERT ON Account
FOR EACH ROW
BEGIN
    -- If customer, create customer record and cart
    IF NEW.role = 'Customer' THEN
        INSERT INTO Customer (customer_id, address, add_phone,
                              total_spent, total_order)
        VALUES (NEW.account_id, NULL, NULL, 0, 0);

        INSERT INTO Cart (customer_id)
        VALUES (NEW.account_id);

    -- If shop, create shop record
    ELSEIF NEW.role = 'Shop' THEN
        INSERT INTO Shop (shop_id, shop_name, shop_phone,
                          address_shop, rating)
        VALUES (NEW.account_id, CONCAT('Shop ', NEW.account_id),
                NULL, NULL, 0);

    -- If admin, create admin record
    ELSEIF NEW.role = 'Admin' THEN
        INSERT INTO Admin (admin_id, role, note)
        VALUES (NEW.account_id, 'Support', NULL);
    END IF;
END$$
```

Trigger Type: AFTER INSERT

Related Tables: ACCOUNT, CUSTOMER, SHOP, ADMIN, CART

Business Logic: Implements inheritance pattern (Account superclass):

1. **Customer Creation**: Creates Customer record + Cart for role='Customer'
2. **Shop Creation**: Creates Shop record with default name for role='Shop'
3. **Admin Creation**: Creates Admin record for role='Admin'
4. **Referential Integrity**: Ensures 1:1 relationship between Account and subclass

Example Scenario:

```
-- Create customer account
INSERT INTO Account (email, password, full_name, phone, role)
VALUES ('customer@email.com', 'hash123', 'John Doe',
       '0123456789', 'Customer');
-- account_id = 10

-- After trigger execution:
-- Customer table: customer_id=10, total_spent=0, total_order=0
-- Cart table: cart_id=10, customer_id=10

-- Create shop account
INSERT INTO Account (email, password, full_name, phone, role)
VALUES ('shop@email.com', 'hash456', 'Shop Owner',
       '0987654321', 'Shop');
-- account_id = 11

-- After trigger execution:
-- Shop table: shop_id=11, shop_name='Shop 11', rating=0
```

10.4.2 Trigger: Customer Before Insert (Code Generation)

Purpose: Auto-generates customer_code with CUST prefix (CUST00001, CUST00002, ...) based on customer_id.

Listing 52: Trigger: trg_customer_before_insert

```
CREATE TRIGGER trg_customer_before_insert
BEFORE INSERT ON Customer
FOR EACH ROW
BEGIN
    -- Generate customer_code from customer_id
    SET NEW.customer_code = CONCAT('CUST', LPAD(NEW.customer_id, 5, '0'));
END$$
```

Trigger Type: BEFORE INSERT

Related Tables: CUSTOMER

Business Logic: Implements auto-generated business key:

1. **Code Format**: CUSTxxxxx where xxxxx is zero-padded customer_id
2. **LPAD Function**: Pads ID with leading zeros to 5 digits
3. **Consistency**: Ensures all customer codes follow same format

Example Scenario:

```
-- Insert customer with ID 1
INSERT INTO Customer (customer_id, address, total_spent, total_order)
VALUES (1, '123 Main St', 0, 0);

-- After trigger: customer_code = 'CUST00001'

-- Insert customer with ID 25
INSERT INTO Customer (customer_id, address, total_spent, total_order)
VALUES (25, '456 Elm St', 0, 0);

-- After trigger: customer_code = 'CUST00025'

-- Insert customer with ID 1000
INSERT INTO Customer (customer_id, address, total_spent, total_order)
VALUES (1000, '789 Oak St', 0, 0);

-- After trigger: customer_code = 'CUST01000'
```

10.4.3 Trigger: Shop Before Insert (Code Generation)

Purpose: Auto-generates shop_code with SHOP prefix (SHOP00001, SHOP00002, ...) based on shop_id.

Listing 53: Trigger: trg_shop_before_insert

```
CREATE TRIGGER trg_shop_before_insert
BEFORE INSERT ON Shop
FOR EACH ROW
BEGIN
    -- Generate shop_code from shop_id
    SET NEW.shop_code = CONCAT('SHOP', LPAD(NEW.shop_id, 5, '0'));
END$$
```

Trigger Type: BEFORE INSERT

Related Tables: SHOP

Business Logic: Same pattern as customer_code generation:

1. **Code Format**: SHOPxxxxx where xxxxx is zero-padded shop_id
2. **LPAD Function**: Pads ID with leading zeros to 5 digits
3. **Business Key**: Provides user-friendly identifier for shops

Example Scenario:

```
-- Insert shop with ID 4
INSERT INTO Shop (shop_id, shop_name, rating)
VALUES (4, 'Tech Store', 0);

-- After trigger: shop_code = 'SHOP00004'

-- Insert shop with ID 100
INSERT INTO Shop (shop_id, shop_name, rating)
VALUES (100, 'Fashion Hub', 0);

-- After trigger: shop_code = 'SHOP00100'

SELECT shop_id, shop_code, shop_name FROM Shop;
-- Results:
-- 4  | SHOP00004 | Tech Store
-- 100| SHOP00100 | Fashion Hub
```

11 Application Demo and System Workflow

This section demonstrates the complete e-commerce system architecture and how the application works from database to user interface.

11.1 System Architecture Overview

11.1.1 Three-Tier Architecture Design

1. Data Layer (MySQL Database): The database layer handles all data storage and business logic execution:

- **14 Tables**: Account (superclass), Customer, Shop, Admin (inheritance), Product, ProductItem, Category (recursive hierarchy), Order, OrderItem, Cart, CartItem, Voucher, Shipping, Review
- **6 Functions**: Calculate totals (order total, customer spent, shop revenue), Calculate ratings (product rating, shop rating), Cursor demonstration (order total items)
- **13 Procedures**:

  - Category hierarchy navigation (get hierarchy, get all children, get all parents) - using Recursive CTE
  - Order management (create order from cart, apply voucher) - complex transactions
  - Shop analytics (revenue report with top products) - aggregations and reporting
  - Product CRUD (add, update, delete product and product items) - data manipulation
  - Product search (dynamic filtering, sorting, pagination) - advanced queries
- **9 Triggers**: Auto-generate customer/shop codes, manage inventory stock, update rating statistics, create cart automatically, enforce business rules

2. Application Layer (Node.js + Express): The backend provides RESTful API endpoints organized by business modules:

- **MVC Architecture**: Controllers handle requests, call database procedures/functions, return JSON responses
- **11 Controller Modules**:
  - `auth.controller.js`: User registration, login, JWT token management
  - `product.controller.js`: Product search, CRUD operations, variant management
  - `cart.controller.js`: Add/remove items, update quantities, view cart
  - `order.controller.js`: Create orders, view history, update status, cancellation
  - `customer.controller.js`: Profile management, order history, statistics
  - `shop.controller.js`: Shop information, revenue reports, analytics
  - `review.controller.js`: Submit reviews, view ratings, calculate averages
  - `category.controller.js`: Category tree, parent-child relationships
  - `voucher.controller.js`: Validate codes, apply discounts
  - `shipping.controller.js`: Shipping methods, fee calculation
  - `admin.controller.js`: System management, user administration
- **Middleware**: JWT authentication, role-based authorization, error handling, request validation
- **Routes**: RESTful endpoints following naming conventions (GET /api/products, POST /api/orders, etc.)

3. Presentation Layer (React + Redux): The frontend provides interactive user interfaces for different user roles:

- **Page Structure**:
  - **Public Pages** (5): HomePage, SignInPage, SignUpPage, ProductListPage, ProductDetailPage

  - **Customer Pages** (6): CartPage, CheckoutPage, OrderConfirmationPage, OrderDetailPage, ProfilePage, ReviewPage
  - **Shop Owner Pages** (4): ShopDashboard, ShopProducts, ShopOrders, ShopAnalytics
  - **Admin Pages** (3): AdminDashboard, UserManagement, SystemReports

- **Components**: Reusable UI components (Button, Modal, Card, Input, Dropdown, Spinner, Toast notifications)
- **State Management**: Redux stores for auth, cart, products, orders - synchronized across all pages
- **Routing**: React Router with protected routes based on user roles

11.2 Main Features Demonstration

11.2.1 Feature 1: Product Search and Discovery

User Experience:

- Customer visits homepage, browses category menu (Electronics > Phones)
- Enters search term "iPhone"in search bar
- Applies filters: Price range (0-2,000,000 VND), Rating (4+ stars)
- Sorts results by rating (descending)
- Views paginated results showing 20 products per page

Behind the Scenes:

- Frontend sends GET request: `/api/products?search=iPhone&category=3&minPrice=0&maxPrice=`
- Backend calls: `CALL sp_get_product_list(...)`
- Database executes dynamic SQL with JOINs (Product, ProductItem, Shop, Review)
- Calculates: MIN/MAX price, total stock, variant count, average rating
- Returns JSON with products array and pagination metadata
- Frontend renders product grid with images, prices, ratings, stock status

11.2.2 Feature 2: Shopping Cart Management

User Experience:

- Customer selects product variant (iPhone 15, Red, 128GB)
- Clicks "Add to Cart"button

- Toast notification confirms: "Product added to cart!"
- Cart icon updates showing item count
- Navigates to CartPage to review items
- Adjusts quantities using +/- buttons
- Removes unwanted items
- Sees real-time subtotal calculation

Behind the Scenes:

- Frontend: POST `/api/cart/add` with `{item_id, quantity}`
- Backend: INSERT into CartItem or UPDATE quantity if exists
- Database: Retrieves cart_id from Cart table (one cart per customer)
- Redux store updates cart state globally
- All components showing cart info re-render automatically

11.2.3 Feature 3: Order Creation with Voucher

User Experience:

- Customer proceeds to checkout
- Selects shipping method (Standard Shipping - 5,000 VND)
- Enters voucher code "DISCOUNT10"and clicks "Apply"
- System validates voucher: "10% discount applied! Save 220,000 VND"
- Confirms shipping address and payment method (Banking)
- Reviews final total: Subtotal (2,200,000) + Shipping (5,000) - Discount (220,000) = 1,985,000 VND
- Clicks "Place Order"
- Redirected to order confirmation page with order ID

Behind the Scenes:

- Voucher validation: `CALL sp_apply_voucher(code, amount, @discount, @valid, @message)`
- Checks: voucher exists, is active, not expired, usage limit not reached, order meets minimum value
- Order creation: `CALL sp_create_order_from_cart(...)`

- Database transaction executes:
  1. Validates customer, shipping method, cart has items
  2. Calculates subtotal from CartItems
  3. Applies voucher discount, adds shipping fee
  4. Inserts Order with status 'Processing'
  5. Moves CartItems to OrderItems
  6. Reduces ProductItem stock (inventory management)
  7. Clears cart
  8. Updates Customer statistics (total_order++, total_spent+=amount)
- Frontend receives order_id, updates Redux, shows confirmation

11.2.4 Feature 4: Product Review and Rating

User Experience:

- Customer receives order with status "Delivered"
- Navigates to OrderDetailPage, sees "Leave Review"button
- Opens ReviewPage for Product 1 (iPhone 15)
- Selects 5-star rating
- Writes comment: "Excellent product! Fast delivery, highly recommend!"
- Clicks "Submit Review"
- Success message: "Review submitted! Product rating updated to 4.67 stars"

Behind the Scenes:

- Frontend: POST `/api/reviews/add` with `{target_id, target_type, rating, comment}`
- Backend: INSERT INTO Review
- **Trigger fires automatically**: `trg_review_after_insert`
- Trigger calls function: `fn_calculate_product_rating(product_id)`
- Function calculates: `AVG(rating)` from all product reviews
- Example: Previous reviews (5, 4), New review (5) → Average = (5+4+5)/3 = 4.67
- Trigger updates: `UPDATE Product SET rating = 4.67`
- All users immediately see updated rating on product pages

11.2.5 Feature 5: Shop Revenue Analytics

User Experience:

- Shop owner logs in, navigates to ShopAnalytics page
- Selects date range: January 1, 2025 - December 31, 2025
- Clicks "Generate Report"
- Views dashboard showing:
  - Total revenue: 1,980,000 VND
  - Total orders: 1
  - Total units sold: 2
  - Average order value: 1,980,000 VND
  - Top 10 selling products with breakdown by variant
- Exports report to PDF for accounting

Behind the Scenes:

- Frontend: GET `/api/shops/4/revenue?from=2025-01-01&to=2025-12-31`
- Backend: `CALL sp_get_shop_revenue_report(shop_id, from_date, to_date)`
- Procedure returns two result sets:
  1. Shop summary with aggregated metrics
  2. Top 10 products ordered by revenue (DESC)
- Frontend renders charts (revenue trend graph, product pie chart)

12 References and Video Demonstration

12.1 GitHub Repository

The complete project source code is available on GitHub: Link Github: Link Github

12.2 Video Demonstration

A comprehensive video demonstration of the application is available. The video covers: Video demo: HERE

13 Conclusion

This report has presented a complete full-stack implementation of the Shoppii e-commerce platform, integrating database design, backend API, and frontend interface.

13.1 Summary

The project demonstrates:

- **Database (MySQL)**: 13 tables, 6 functions, 13 procedures, 9 triggers with advanced SQL techniques (Recursive CTE, transactions, automatic triggers)
- **Backend (Node.js/Express)**: 11 controllers with 50+ RESTful endpoints, JWT authentication, role-based access control
- **Frontend (React/Vite)**: 19+ pages, Redux state management, real-time updates, responsive UI components
- **Integration**: Seamless workflow from user interface through API to database procedures with automatic trigger coordination

13.2 Key Features

- Multi-role platform: Customers, Shop Owners, Admins
- Product search with dynamic filtering and sorting
- Shopping cart and checkout with voucher support
- Order processing with automatic stock management
- Review system with automatic rating calculation
- Shop analytics and revenue reporting
- Transaction safety with ACID compliance